

Portfolio and Applied Labs Complete Project Manual

Project: Deepan Karthick's portfolio upgrade

Updated: 2026-09-04. This handbook explains how to operate, understand, test, and maintain two flagship business applications and 30 certificate-linked Applied Labs. The portfolio redesign remains local work pending release.

For reviewers and maintainers: each project chapter covers its user problem, inputs, rules, worked examples, source/test map, privacy boundaries, and limitations. Use the setup and architecture sections first; use the appendix to locate a specific route or test.

Current change scope: portfolio only, including its public documentation. The user explicitly excluded Timber and Printing Press ERP; Applied Labs is also outside this design pass. Their chapters describe existing behavior, not authorization to redesign, repair, rebuild, or redeploy those applications. General multi-surface maintenance instructions below apply only when that broader work is separately in scope.

Evidence and privacy: current source, specifications, tests, manifest, and dated batch records support the documented behavior. Use synthetic data only; never publish customer records, databases, imported runtime bundles, or secrets. Historical passes do not establish current production health.

Existing release: <https://deepan-karthick-portfolio.industrious-keyboard.workers.dev/> on Cloudflare Workers Static Assets, with Applied Labs at [/labs/](#) and the sanitized Timber demo at [/timber-demo/](#). Printing ERP is a separate deployment at <https://lakshmipriya-erp-demo.industrious-keyboard.workers.dev/owner>. This correction does not constitute a new deployment or live health check.

Artifact freshness: PDF and DOCX were regenerated from this expanded source on 2026-09-04. PDF rendering is checked separately from source tests. DOCX visual acceptance remains pending because the required LibreOffice renderer is unavailable. Generation does not publish the files or prove application acceptance.

Release boundary: production publication and live-browser acceptance remain pending. Follow [docs/project-control/STATUS.md](#) for the latest dated checkpoint; the original guide-coverage plan is historical, not a release record.

Documentation checks: the canonical manifest, 32 project chapters, 90 appendix file references, README local links, and nine Timber domain tests passed on 2026-09-04. Earlier lab example test evidence is dated in the chapters. Browser and screenshot acceptance remain separate.

1 What this project is

This mission turns a list of 30 completed credentials into evidence of applied work. The portfolio homepage is intended to feature only two substantial flagship applications:

- 1 **Printing Press ERP** - a public, synthetic, browser-local demonstration of a printing-business workflow, with a separately protected live mode for the private system.
- 2 **Timber CFT Pro with Billing** - a sanitized timber measurement and invoice application that preserves the supplied business rules without publishing client data.

The remaining 30 certificate-linked projects live in one **Applied Labs** application. Each lab has its own stable route but shares one catalog, one visual shell, and one deployment surface. The labs are not certificate thumbnails or generic quizzes. Each is a small working tool for a named or clearly identifiable user, with realistic inputs, deterministic decision logic, a useful output, validation, sample/reset controls, tests, and an honest limitation statement.

What functional first means

The project deliberately separates correctness from final visual polish. The implementation order is:

- 1 lock scope, privacy, and business rules;
- 2 create a realistic input and useful output;
- 3 make the workflow deterministic and inspectable;
- 4 add validation, example data, reset, and copy/download/print handoff where relevant;
- 5 test the pure domain logic;
- 6 integrate a lazy route and verify the rendered workflow;
- 7 only after all functional batches, redesign the portfolio and labs visually;
- 8 then update resumes, deploy, run production regression checks, and finish user documentation.

This order reduces the chance that visual work hides incomplete behavior. It also allows each batch to be tested and repaired before more projects are added.

Current source state and evidence boundaries

Area	Recorded state	Important boundary
Printing Press ERP	Existing separate Cloudflare public demo; earlier batch report records functional acceptance	Private source is outside this public source tree; current dependency health was not rechecked in this documentation pass
Timber CFT Pro with Billing	Sanitized static React demo with measurement, billing, and browser-local history; packaged at /timber-demo/; screenshot replacement made locally	Application is excluded from current changes; portfolio screenshot/link publication remains pending
Labs 01-24 and 26	Manifest-marked functional; earlier batch reports contain automated/build/browser evidence	The new visual redesign requires fresh route-level regression; old passes do not cover new changes
Labs 25 and 27-30	Implemented, documented, lazy-route integrated, and manifest-marked functional; earlier 151-test checkpoint recorded	Consolidated later acceptance records were not closed in the batch-report directory; reconcile them with fresh regression evidence
Portfolio homepage	Two-flagship animated redesign exists in local source; existing unified Cloudflare release remains separate	Current scope is portfolio-only completion and verification; release remains pending

The root `README.md` now introduces the portfolio mission and retains rental research only as a legacy utility. The manifest supplies canonical project names/slugs; current source/tests supply exact behavior; dated batch/release evidence supplies verification claims. Where records disagree, state the conflict rather than treating a completion label or HTTP response as end-to-end acceptance.

2 Delivery strategy for a large project

The work is split into bounded batches so failures stay local and the computer is not asked to build everything concurrently.

Batch	Scope	Recorded checkpoint
0	repository control plane, manifest, Applied Labs shell	architecture and validation foundation
1A	Timber CFT Pro with Billing	9 Timber tests; combined build and browser flow passed
1B	Printing Press ERP repair and Cloudflare deployment	6 runtime/store tests, lint, build, browser routes, persistence, and clean console passed
2	Labs 01-05	30 cumulative Node tests; builds and browser acceptance passed
3	Labs 06-10	53 cumulative tests; builds and browser acceptance passed
4	Labs 11-14 and 26; Lab 03 usefulness repair	78 cumulative tests; builds and browser acceptance passed
5	Labs 15-19	104 cumulative tests; builds and browser acceptance passed
6	Labs 20-24	126 cumulative tests; builds and browser acceptance passed
7	Labs 25 and 27-30	Historical 151-test checkpoint; consolidated build/browser acceptance record needs reconciliation

Batch	Scope	Recorded checkpoint
8	original portfolio and labs visual redesign	Existing source baseline; current continuation is portfolio-only, pending release
9	ATS and creative/startup resume refresh	Existing generated outputs; not regenerated or revalidated in this documentation pass
10	original unified release and manual	Existing Cloudflare release and generated manual; current Markdown corrections have not been regenerated or redeployed

Parallel agents may be used only on isolated, non-overlapping files. Shared routing, manifests, styles, status, and deployment are coordinator-owned integration points. The practical rule is: one implementation batch at a time, a focused test after each lab, then one central test/build/browser gate.

The redesign and detailed guide use separate checkpoints so earlier functional batches are not mistaken for acceptance of new work. Detailed Markdown chapters now cover 32 projects. The original coverage matrix and D0-D8 sequence are historical planning material, not current completion status; final screenshots, regenerated artifacts, rendered QA, and published-link acceptance remain pending.

3 Architecture and repository map

project/	
├─ portfolio-src/	Portfolio source (Vite root)
│ ├─ index.html	Main portfolio entry
│ ├─ links.html	Links entry
│ ├─ dashboard.html	Dashboard entry
│ ├─ main.jsx	Main React portfolio source
│ └─ public/	Static portfolio assets/data
├─ labs-src/	Applied Labs source (separate Vite app)
│ ├─ main.jsx	Catalog, filters, lazy routes, lab registry
│ ├─ styles.css	Shared Applied Labs presentation
│ └─ labs/	Pure domain modules + React lab components
├─ timber-src/	Sanitized Timber public demo source
│ ├─ App.jsx	Invoice workflow
│ ├─ domain/calculations.js	Locked measurement rules
│ ├─ domain/invoice.js	Billing and validation rules
│ └─ storage.js	Browser-local persistence
├─ tests/	Node domain tests for Timber and Labs 01-30
├─ scripts/	
│ ├─ validate_project_manifest.mjs	
│ ├─ assemble_cloudflare_site.mjs	Unified static package assembly
│ └─ build_complete_manual.py	Markdown to DOCX/PDF generation
├─ docs/project-control/	Decisions, specs, status, lab and batch records
├─ portfolio/	Generated portfolio build output
├─ labs/	Generated Applied Labs build output
├─ timber-demo/	Generated Timber build output
├─ site-dist/	Assembled Cloudflare package (generated)
├─ wrangler.jsonc	Worker name and static-assets directory
├─ vite.config.js	Portfolio multi-page build
├─ vite.labs.config.js	Applied Labs build
├─ vite.timber.config.js	Timber demo build
└─ package.json	Local scripts and dependencies

The Printing Press ERP source is recorded as living in a separate private `lakshmipriya-erp` repository. This repository contains the specification, verification report, portfolio screenshot, and deployed URL, but not the private ERP source tree. That separation should be preserved.

Runtime design

- **Portfolio:** React 19 + Vite 6, with Framer Motion, GSAP, and React Spring in the existing source.
- **Applied Labs:** React/Vite single-page application using hash routes such as `#/lab/mini-rag-studio`. Each lab component is imported lazily, so one lab does not force all lab components into the initial route chunk.
- **Timber demo:** separate React/Vite static application. It has no required server and stores synthetic invoices in the current browser.
- **Printing ERP demo:** standalone application deployed with Cloudflare Workers Static Assets. Its public demo runtime is browser-local and must create no Supabase client or request.
- **Domain logic:** labs generally separate pure `.js` functions from `.jsx` presentation, allowing Node tests without a browser.
- **Network boundary:** the core lab calculations are local and do not require API keys, model calls, cloud accounts, wallets, scanners, or paid services.

4 Local setup run test and build

Prerequisites

- Node.js and pnpm must be installed.
- The existing frontend CI uses Node 22 and pnpm 11.19.0. `package.json` does not pin an `engines` field; use the CI configuration as the recorded reproducibility baseline and confirm with the full `test/build` commands.
- No environment variable is required for the portfolio, Applied Labs, or public Timber demo.
- Do not place private SQLite databases, real customer exports, credentials, or ERP live-mode secrets in this repository.

Install

```
pnpm install --frozen-lockfile
```

The lockfile is `pnpm-lock.yaml`. The frozen install above matches `.github/workflows/frontend-tests.yml`. Deliberate dependency updates should update and review the lockfile separately.

Development servers

Run one surface at a time unless there is a specific integration reason to run more:

```
pnpm run dev:portfolio
pnpm run dev:labs
pnpm run dev:timber
```

`pnpm run dev` is an alias for `dev:portfolio`. Each command binds to `127.0.0.1`; use the port Vite prints in the terminal.

Tests

```
pnpm run test
```

This first validates that the canonical manifest contains the expected 30 labs and 2 flagships, then runs all `tests/*.test.mjs` files with Node's built-in test runner. An older Python research-prototype test is outside this public-surface command.

Run a focused lab test while developing:

```
node --test tests/lab24-feature-misuse-contract.test.mjs
```

Replace the filename with the lab being changed. Timber's focused domain suite is:

```
node --test tests/timber-domain.test.mjs
```

Production builds

```
pnpm run build
```

The combined command runs tests first and then creates all three public build outputs:

- `pnpm run build:portfolio → portfolio/`
- `pnpm run build:labs → labs/`
- `pnpm run build:timber → timber-demo/`

For the actual unified Cloudflare package, run:

```
pnpm run build:site
```

This runs the combined test/build pipeline, then `scripts/assemble_cloudflare_site.mjs`. The assembly places the portfolio at the root of `site-dist/`, retains a `/portfolio/` compatibility copy, adds `/labs/` and `/timber-demo/`, and copies the selected resume/manual PDFs into `/docs/generated/`. `pnpm run site:assemble` only repackages existing build outputs; it does not rebuild or test them. `wrangler.jsonc` points the `deepan-karthick-portfolio` Worker at `./site-dist`.

The Node pipeline does not regenerate the manual. `scripts/build_complete_manual.py` reads this Markdown and generates DOCX/PDF using `python-docx` and `ReportLab`. The September 4 artifacts contain the expanded chapters. Re-run generation and visual checks after later source changes; a successful site build does not establish rendered-document acceptance. DOCX visual QA is currently blocked by the missing LibreOffice runtime.

Preview the built surfaces with:

```
pnpm run preview
pnpm run preview:labs
pnpm run preview:timber
```

The Printing ERP has its own private repository, commands, and deployment configuration. Those commands are not present in this repository and should not be guessed.

5 Flagship 1 Printing Press ERP

Why it was built users and the real problem

A printing press does not need isolated dashboard mockups; it needs a shared workflow from customer order through design, production, dispatch, invoice, and management evidence. The owner needs business-wide status and exception visibility. Managers coordinate staff, orders, inventory, and production. Designers need assigned work and upload handoff. Accounts staff need invoice/payment views. A public portfolio reviewer needs a convincing workflow without seeing the real company's customers or backend. The flagship was therefore built as one role-routed ERP with an explicit synthetic demonstration mode rather than publishing a private operational database.

Architecture and component map

The audited private source is a React 19/Vite single-page application using React Router, lazy-loaded route modules, a top-level error boundary, and Suspense loading fallback. `AppRoutes` maps public landing/gallery/login pages and protected owner, manager, staff, order, production, notification, inventory, reverse-request, report, designer, and accounts routes. `AuthProvider` chooses the runtime-specific authentication implementation. `ProtectedRoute` requires a user; `RoleRoute` normalizes the profile role and redirects unauthorized users to their own dashboard. Page hooks coordinate loading/error/refresh state; service modules own orders, dashboards, inventory, invoices, notifications, designs, reports, and staff operations. PDF export is an explicit utility rather than page arithmetic.

The two data paths are deliberately different:

Public demo: route/page → hook → service demo branch → versioned localStorage seed/store

Private live: route/page → role/auth guard → service → Supabase client → governed tables/storage

`VITE_APP_MODE` resolves to `live` only on an explicit case-normalized `live` value; blank or unexpected values resolve to `demo`. Demo authentication is a synthetic owner profile. The Supabase client is constructed only when live mode is selected and the expected URL/key relationship passes configuration checks; otherwise a non-network demo boundary is used. Live configuration failure renders a bounded configuration screen rather than an endless load.

Exact public demo domain rules

The demo store is versioned under one browser-local key. Missing, malformed, wrong-version, or non-array state resets to five synthetic seed orders. Reads are cloned and sorted newest first. A created order gets an opaque ID, synthetic client ID, current timestamps, `Pending` stage, `pending` status, `normal` priority, and an order number `LP-YYYY-NNNN` using the highest sequence for the current year plus one. Creation requires a company name, product type, quantity greater than zero, a delivery date, a 10-digit WhatsApp field, and an optional 10-digit client phone. Stage changes map `Completed` → `completed`, `Ready Dispatch` → `ready_dispatch`, and all other production stages to `in_progress`. Supported production labels in source are `Pending`, `Design`, `Plate Making`, `Printing`, `Lamination`, `Dye Cutting`, `Nurbing`, `Pasting`, `Greasing`, `Ready Dispatch`, and `Completed`. Priority values are `urgent`, `normal`, and `delayed`.

Role routing is concrete: owner has owner-only reverse requests and reports plus broad operational access; manager shares staff/order/production/notification/inventory access; designer receives designer dashboard/upload; accounts receives accounts dashboard/invoices. Staff-management rules prohibit disabling an owner or changing the owner role through the ordinary staff screen. These UI/service rules complement-rather than replace-database row-level security in live mode.

End to end synthetic workflow

- 1 Open /owner; confirm the five fictional seed orders and dashboard counts render.
- 2 Open the production board; inspect stage, status, delivery, and priority rather than relying only on a total.
- 3 Create an order with synthetic values such as company "Example Demo Client," product "Sample carton," material "Demo paper stock," quantity 250, A4, and a future date.
- 4 Confirm the order list grows from five to six and the new number matches LP-YYYY-NNNN.
- 5 Reopen the record; move it to Printing and confirm status becomes in_progress; set a priority and reload.
- 6 Visit staff, inventory, notifications, designs, accounts/invoices, reverse requests, and reports under the role allowed by the route map.
- 7 Reset the demo store and confirm it returns to the five synthetic seed orders.

This workflow demonstrates coordination and local persistence. It does not demonstrate multi-user concurrency, live notifications, database policies, or the health of the private backend.

Privacy authentication and data boundaries

The public build must contain only fictional companies, contacts, orders, invoices, inventory, staff, and design metadata. Reviewer-created records remain in that browser and may be cleared. Never place real names, phone/WhatsApp numbers, GST identifiers, invoice files, uploaded designs, Supabase keys, sessions, schema exports, or customer records in the public repository, screenshot, guide, seed, or test fixture. An anon key is configuration, not a substitute for row-level security; no service-role key belongs in a browser build.

The intended boundary is that live failure never masquerades as demo success. Source review found a qualification that must stay visible: several live service catch paths still return synthetic objects when Supabase reports a missing-table condition. Those compatibility fallbacks do not activate the versioned browser demo store, but they can conceal an incomplete live schema. Until removed or explicitly constrained and covered by live-mode tests, the manual must not claim that every live backend failure is universally fail-closed. Authentication/configuration and role routing do fail closed at their inspected boundaries; service-level missing-table behavior remains maintenance work.

Setup build test deploy and rollback

Keep the private ERP checkout and its environment outside the public portfolio package. Use a supported Node version (the audited package declares Node $\geq 20.19.0$ <25), install exactly from its lockfile, keep local secrets in the ignored environment mechanism, and use the package scripts for test, lint, build, dev, and preview. Select demo/live deliberately; do not let a default demo build substitute for live acceptance. Before a live build, verify the expected Supabase project, anon-key/project relationship, authenticated profile lookup, disabled-account denial, every role route, and row-level policies in the governed backend.

The public demo build emits static `dist` assets. Its Cloudflare Workers Static Assets configuration uses SPA not-found handling so direct React routes return the application shell. Deployment must follow the private repository runbook; this public manual does not invent a missing deploy script or expose configuration values. Acceptance requires `/owner`, a direct nested route, `create/reopen/stage/priority/reload/reset`, secondary routes, mobile layout, console, and network inspection proving demo mode makes no Supabase request.

Rollback is release rollback, not data rollback: retain the previously accepted commit/artifact and Cloudflare version, redeploy or restore that known version if route or runtime checks fail, and independently assess any live database migration before application rollback. Never roll a private schema backward merely because a static front end was rolled back. The recorded public URL is <https://lakshmipriya-erp-demo.industrious-keyboard.workers.dev/owner>; historical version `4bd9ebd0-69d8-4c3f-b238-947fde815cc5` and commit `6048565` are provenance, not current health.

Test evidence troubleshooting and maintenance

The documentation lane reran the six available isolated domain tests on 2026-09-01: two runtime-mode tests and four demo-store tests all passed. They prove safe default/explicit live selection, five-order seed, create persistence, stage/status persistence, and corrupt-state recovery. Historical Batch 1B additionally records ESLint, a 2,451-module build, direct-route Cloudflare fallback, core route/browser checks, persisted priority, and clean accepted console. Native-date create submission remained a browser-automation gap; do not convert unit coverage into a browser pass.

- Blank dashboard in demo: confirm the build is actually demo and clear/reset corrupt local demo state.
- Redirect loop: inspect normalized profile role and route allowlist; do not add an owner bypass.
- Live configuration screen: correct the private environment/project pairing and redeploy; never paste a key into the UI.
- Live data unexpectedly looks synthetic: treat it as a release blocker and inspect missing-table compatibility catches plus schema/migrations.
- Direct nested route returns 404: verify SPA not-found handling in the ERP's own Cloudflare config.
- A stage changes without expected log/notification: verify the primary order update and each secondary evidence insert; do not report full workflow completion from only the returned UI object.

Future separately authorized ERP maintenance should add explicit live-mode failure tests, remove or narrowly govern missing-table synthetic fallbacks, rerun the dependency audit (the historical report found two moderate and five high advisories), test row-level policies/role matrices, exercise create-order in a browser path that supports native dates, and establish external health monitoring if availability will be claimed. ERP is excluded from the current change scope. Interview explanation: “I separated a synthetic reviewer experience from an authenticated role-based ERP, built persistent order/production workflows, and can explain the remaining live-service fallback risk rather than hiding it.” Existing synthetic project screenshot publication remains pending; no ERP redesign is planned in this pass.

6 Flagship 2 Timber CFT Pro with Billing

Problem and users

The application serves a timber business that measures pieces using its established “business inch” convention and needs those measurements converted into invoice quantities and financial totals. The public portfolio user can operate a complete synthetic invoice without receiving or exposing the client's Windows/SQLite data.

Product editions

- **Private Windows edition:** supplied Express/SQLite application for offline client use.
- **Public portfolio demo:** sanitized static React application with synthetic sample data and browser-local storage only.

The private SQLite database and bundled runtime must never be committed or deployed. Repository ignore rules cover `.db`, `.db-wal`, and `.db-shm`.

Locked measurement rules

These are business rules, not ordinary physical conversions, and must not be “corrected” without client approval and golden-test changes:

- 1 Business inches = millimetres ÷ 20, not ÷ 25.4.
- 2 A positive dimension rounds upward to the next 0.5 inch.
- 3 The minimum positive billing dimension is 1.5 inches.
- 4 $\text{CFT} = \text{rounded thickness inches} \times \text{rounded width inches} \times \text{length feet} \times \text{pieces} \div 144$.
- 5 $\text{ICBM} = \text{CFT} \div 35.315$.
- 6 M3 uses raw millimetres and actual length in metres.
- 7 PF length may be entered in feet or millimetres.

The calculation rules live in `timber-src/domain/calculations.js` and are protected by `tests/timber-domain.test.mjs`.

Billing rules

The invoice accepts bill number, party/customer, date, lorry number, item type, CFT or M3 pricing basis, rate, discount, GST percentage, and paid amount.

```

row amount      = selected measurement quantity × rate
subtotal        = sum of rounded row amounts
taxable amount  = max(0, subtotal - discount)
GST             = taxable amount × GST percentage
grand total     = taxable amount + GST
balance        = max(0, grand total - paid)
change due      = max(0, paid - grand total)

```

Rate, discount, and paid amount must be non-negative. GST is bounded from 0 to 100. Measurement values and piece counts have strict safe-demo bounds; zero, negative, non-integer piece counts, and unsafe ranges are rejected.

User workflow

- 1 Load the synthetic example or start a blank invoice.
- 2 Enter bill and customer metadata using synthetic values in the public demo.
- 3 Add, edit, or remove measurement rows.
- 4 Observe CFT, M3, row amount, and invoice totals update.
- 5 Correct validation errors before saving.
- 6 Save to browser-local history.
- 7 Search and reopen the invoice.
- 8 Delete only after confirmation.
- 9 Print or use the browser's Save as PDF action.

Privacy and storage

The public demo has no account, server database, analytics requirement, or cloud synchronization. Its invoices stay in the current browser under `deepan.timber-cft-demo.invoices.v1`. An absent key means empty history. Invalid JSON or a non-array value raises `StorageDataError`; reads and saves do not silently overwrite it. The UI displays an alert and a separately confirmed **Clear damaged local history** action. Clearing is irreversible; there is no built-in recovery/export service. Preserve any needed local data before choosing it. The interface tells users to use synthetic data.

Input and output reference

Input	Contract in the public demo
Bill number, party, date	Required to save/print. Bill number and party are trimmed for requiredness. UI text limits: 80 and 140 characters. Date defaults to the local calendar date.
Lorry number, item / wood type	Optional synthetic metadata; UI text limits 80 and 120 characters.
T (thickness), L (width), PF length	Finite positive numbers, at most 1,000,000 in the entered unit. T and L are millimetres; PF unit is ft or mm.
Pieces	Positive integer, at most 100,000. Fractional counts are invalid.
Pricing basis and rate	CFT or M3. UI requires rate greater than zero and at most INR 100,000,000.
Discount, GST, paid	Finite values; discount and paid cannot be negative. Save/print rejects a discount above subtotal. GST must be 0-100 percent.
Row amount	Finite and at most INR 1,000,000,000,000; an invalid/unsafe row contributes zero billed amount and an error.

The domain calculation normalizes negative money inputs and clamps discount/tax to safe arithmetic bounds; the form applies stricter save/print validation. A displayed clamped preview is not permission to save an invalid form. A fully blank row must be completed or removed. Metadata text limits are UI attributes, not a general-purpose import/schema validator.

Quantity display uses three decimal places, but pricing uses unrounded measurement values. `roundMoney` rounds to two decimal places after each row amount, subtotal, applied discount, taxable amount, tax, total, balance, and change due. Total pieces excludes invalid measurement rows. CFT is the business-billing volume; M3 uses original metric dimensions; ICBM is the separate domain value $\text{CFT} \div 35.315$, not a promise that the two volume conventions coincide.

Worked example reproduce the golden invoice

Start blank and enter bill DEMO-GOLDEN-001, party Example Timber Demo, the current date, CFT pricing, rate 1000, discount 100, GST 18, and paid 200. Add one row: T 40 mm, L 60 mm, PF 10 ft, pieces 2.

- 1 Business dimensions are $40 \div 20 = 2$ inches and $60 \div 20 = 3$ inches; neither needs further rounding.
- 2 $CFT = 2 \times 3 \times 10 \times 2 \div 144 = 0.8333333333333334$.
- 3 Metric length = $10 \times 0.3048 = 3.048$ m; $M3 = 40 \times 60 \times 3.048 \times 2 \div 1,000,000 = 0.0146304$.
- 4 Row/subtotal = $\text{round}(0.8333333333333334 \times 1000, 2) = \text{INR } 833.33$.
- 5 Taxable = $833.33 - 100 = \text{INR } 733.33$; GST = $\text{round}(733.33 \times 18 \div 100, 2) = \text{INR } 132.00$.
- 6 Grand total = $\text{INR } 865.33$; balance = $\text{INR } 665.33$; change due = zero.

Changing PF to 3048 mm preserves the volume. For the same row, M3 pricing at 50,000, no discount/tax, and paid 1000 produces $\text{INR } 731.52$ total, zero balance, and $\text{INR } 268.48$ change. Entering thickness zero produces **Thickness must be greater than zero.**; pieces 1.5 produces **Pieces must be a positive whole number.** Neither is a billable row. Entering discount 5000 clamps the domain total to zero, but the UI refuses save/print until the discount is corrected to at most subtotal.

Implementation map and reconstruction sequence

App.jsx owns draft/header/rows, field labels, sample/reset confirmation, validation, history search, status/error messages, and print. domain/calculations.js owns finite-number conversion, billing rounding, measurement validation, and CFT/ICBM/M3. domain/invoice.js owns pricing and financial rounding. storage.js owns versioned browser-local list/save/delete/clear; saving updates the same ID, preserves creation time, and records a new update time. History sorts newest update first; search is case-insensitive across bill, party, lorry, item, and date. Reopening normalizes missing editor defaults and recomputes totals from rows rather than treating a saved summary as new domain truth.

To reconstruct the design without importing the client's runtime: first implement the locked measurement helpers and golden vectors; then add bounded invoice pricing and rounding; then build the controlled React editor and stricter save/print validation; finally add browser-local history with corruption protection and synthetic-only examples. This is reconstruction guidance, not a new application change or a claimed historical commit sequence. Use `node --test tests/timber-domain.test.mjs` to verify the pure domain. Timber development/build commands in §4 are reference only while this application is excluded from current changes.

Operating and troubleshooting details

Load sample replaces the current editor only after confirmation. **Save locally** validates then updates/inserts a history record. **Reset** confirms loss of the current draft but does not clear saved history. **Reopen** also confirms replacement of the editor. **Delete** permanently removes the chosen saved invoice after confirmation; deleting the currently open invoice also resets the editor. **Print / Save PDF** validates and opens the browser dialog; the application does not automatically create a PDF file.

- Unexpected CFT: inspect upward half-inch rounding and the 1.5-inch minimum; never replace $\div 20$ with $\div 25.4$.
- Correct quantity but unexpected rupees: check selected CFT/M3 basis, unrounded priced quantity, per-row money rounding, applied discount, and GST order.

- Unable to save/print: remove unfinished rows and correct metadata, numeric, discount, and row safety errors first.
- Save fails despite valid input: check browser storage availability/quota and the displayed error; the app has no server fallback.
- Empty search: clear the search and confirm the same browser profile/origin; a deployed origin and localhost have different history.
- Damaged history: do not repeatedly save or assume reset repairs storage; read the alert and preserve any needed data before explicitly clearing it.

The existing nine domain tests cover rounding vectors, raw-M3 and PF conversions, malformed/unsafe measurement input, billing totals, invalid rows, capped discount, and overpayment. They do not directly test DOM behavior, storage corruption, print layout, or multi-tab conflicts. Those remain separate browser/storage acceptance concerns; there is no cross-device or transactional multi-user history guarantee.

Verification evidence

The Batch 1A report records 9 Timber tests, manifest validity, and a passing combined production build. The realistic browser sample produced:

- 12 pieces;
- 15.834 CFT;
- 0.240 M3;
- INR 34,270.96 grand total.

Browser-local save, reset, and reopen restored the same invoice and totals. These are verification fixtures, not production usage metrics.

Deployment and limitations

The sanitized static output is `timber-demo/`, assembled into the unified Cloudflare package at `/timber-demo/`. The current portfolio source links to `../timber-demo/index.html`; the public route is `https://deepan-karthick-portfolio.industrious-keyboard.workers.dev/timber-demo/`. Batch 1A's original hosting deferral is historical, not the current deployment topology. The previous manual's `calculator00.pages.dev` association is not the source-of-truth route for this application.

The user reported that the portfolio's Timber image did not show the correct project. The coordinator replaced it locally with an actual project `.jpg` screenshot. Publication remains pending. Timber itself is excluded from the current redesign; retain an actual synthetic-data project screenshot rather than substituting a design concept as evidence. No fresh Timber production workflow acceptance is claimed by this documentation update.

Other limitations:

- browser-local records are not shared or backed up;
- print layout depends on the browser's print engine;
- business formula changes require new approved golden vectors;
- the public demo is not authorization to migrate real client records;

- Timber UI changes are explicitly outside the current portfolio-only scope.

7 Applied Labs platform and UX

Applied Labs is an existing application outside the current portfolio-only design pass. Pending screenshot and browser-regression notes in the lab chapters preserve earlier documentation/acceptance backlog; they do not require or authorize a new lab redesign now.

The Applied Labs application is a catalog rather than 30 separately maintained websites. Its main source is `labs-src/main.jsx`.

Catalog workflow

- Browse all 30 manifest-backed project cards.
- Search by title, credential, or relevant catalog text.
- Filter by AI/ML, cloud, programming/data, business/finance, or cybersecurity.
- Open a stable hash route: `#/lab/<slug>`.
- Return to the catalog or main portfolio.
- While a lab chunk loads, React Suspense provides a loading state.

The catalog and routes are derived from the canonical manifest/registry, while every lab's domain logic remains in its own module. Historical batch reports document browser checks for the first 24 labs and Lab 26, desktop and 390 × 844 catalog checks, and a cybersecurity filter returning seven labs. The later consolidated acceptance record is incomplete; all changed routes need fresh checks for the new redesign regardless of earlier results.

Shared UX contract for every lab

Each lab should provide:

- a clear real user and problem;
- bounded, labelled inputs with useful guidance;
- a realistic example and a reset action;
- validation that withholds unsafe or meaningless output;
- visible decision rules or calculations;
- a useful artifact such as a plan, memo, contract, code starter, test set, export, or comparison;
- privacy and safety boundaries near the workflow;
- keyboard-usable controls and a usable narrow-screen layout;
- a focused Node test for the domain module;
- an honest statement of what the lab cannot prove.

Route convention

Local and deployed routes use the hash form:

`<Applied Labs base URL>#/lab/<slug>`

The public base is

<https://deepan-karthick-portfolio.industrious-keyboard.workers.dev/labs/>. For example, Mini RAG Studio uses <https://deepan-karthick-portfolio.industrious-keyboard.workers.dev/labs/#/lab/mini-rag-studio>. The hash selects the lab in the browser; use the manifest slug and verify the exact route during release QA.

8 Labs 01 10 AI and machine learning

Lab 01 AI Workflow Canvas

- **Certificate connection:** Generative AI for Everyone.
- **Real user/problem:** a learner or small project planner has an AI idea but needs to turn it into an ordered, reviewable workflow rather than an unsupported “add AI” statement.
- **Why built:** to demonstrate that useful AI planning begins with data, task, review, and risk boundaries.
- **Inputs:** goal, data source, task, and human-review level.
- **Decision logic:** validates required context, selects and orders seven workflow steps, adjusts review controls, and identifies overreach/data/quality risks.
- **Outputs/artifacts:** ordered workflow, risk register, and readiness checklist.
- **Safety/privacy:** local deterministic planning; no model, upload, API, production data inspection, or readiness certification.
- **Tests:** `tests/lab01-ai-workflow-canvas.test.mjs` covers the realistic sample, required fields, step ordering, review behavior, and overreach controls.
- **Limitations:** generated steps are a planning starter; they do not prove data quality, model suitability, permission, cost, or production readiness.
- **Route/files:** `#/lab/ai-workflow-canvas`; `aiWorkflowCanvas.js`; `AIWorkflowCanvas.jsx`.

Operate it with the built in example

- 1 Open the route and select **Load example**. This fills the form and builds the result immediately; a second submit is unnecessary.
- 2 The exact goal is **Turn weekly support tickets into a concise product-trend briefing for the product team**. The source description is **An exported CSV of anonymized customer support tickets with issue, product area, and resolution fields**. Task is **summarize**; review is **high** (Expert approval before use).
- 3 Read the seven ordered steps: **Define success** → **Inspect the source** → **Prepare the input** → **Run the AI task** → **Check the result** → **Apply human review** → **Release and learn**.
- 4 Inspect the four risks: **Sensitive data exposure** (High), **Unsupported output** (High), **Automation bias** (Medium), and **Weak or stale input** (Medium). The example still triggers sensitivity because its description contains the word **customer**; the app does not inspect whether anonymization actually succeeded.
- 5 Change the task to **Recommend a next action** and select **Build workflow**. A fifth risk, **Decision overreach**, is added. Changing form fields alone does not rebuild the old result.
- 6 Select **Reset** to clear both form and result. The result is an on-screen planning artifact; this component has no dedicated download, save, or clipboard control.

Input and decision reference

Input	Accepted source behavior	Important effect
Goal	Trimmed text, minimum 12 characters	Embedded in the success step and summary
Data source	Trimmed text, minimum 12 characters	Embedded in inspection guidance; keyword sensitivity check
AI task	classify, summarize, extract, draft, recommend	Selects task wording; recommend adds overreach risk
Human review	low, medium, high	Spot-check / every-output review / expert approval wording

`validateWorkflowInput` returns a keyed error object. Invalid input makes `buildAIWorkflow` return empty steps, risks, and checklist arrays; the UI hides the successful-result region. There is no numeric readiness score and no upper text-length limit in this domain validator. Do not describe it as a hardened large-input processor.

The seven-step order and six checklist items are fixed templates. The only risk branches are: sensitivity keywords in the source description; automation bias is High for low review and Medium otherwise; recommendations add High decision overreach. Sensitivity uses a case-insensitive expression matching customer, personal, private, confidential, patient, employee, email, phone, or financial. It is a warning heuristic, not a privacy classifier.

Rebuild and understand the architecture

Start with the task/review label maps, then implement required-field validation, then construct the fixed ordered steps and conditional risk entries. Keep this in `labs-src/labs/aiWorkflowCanvas.js` so it can be tested without React. In `AIWorkflowCanvas.jsx`, keep form and result as separate state: submit calls the domain function, Load example updates both, Reset clears both. This separation makes explicit when the displayed plan was computed. Register the existing lazy route through `labs-src/main.jsx`, not by adding network services.

Run node `--test tests/lab01-ai-workflow-canvas.test.mjs`. The three focused tests verify sample ordering/review/sensitivity, all four missing-decision errors, and recommendation overreach. They passed in the 2026-08-31 documentation subbatch; they do not verify an eventual redesigned browser layout.

Troubleshooting and honest explanation

If output appears unchanged after editing, press Build workflow. If a form looks complete but fails, count characters after trimming and select actual supported choices. A sensitivity warning may remain for anonymized customer data because matching is lexical. A workflow without that warning is not proof the input is safe. Interview explanation: “I separated planning templates from React state and made human accountability explicit; the tool creates a reviewable process, not AI output.” Final-UI screenshot: **capture pending after redesign acceptance.**

Lab 02 Mini RAG Studio

- **Certificate connection:** Generative AI with Large Language Models.
- **Real user/problem:** a learner or analyst wants to understand how retrieval supplies evidence for an answer without sending a document to a hosted model.
- **Why built:** to make chunking, retrieval, ranking, extraction, and citations inspectable.

- **Inputs:** a bounded pasted corpus and a question.
- **Decision logic:** makes overlapping chunks, tokenizes text, ranks chunks by transparent lexical overlap with deterministic tie-breaking, and extracts evidence-bearing sentences.
- **Outputs/artifacts:** extractive answer, ranked evidence, relevance scores, and chunk citations.
- **Safety/privacy:** runs locally; no upload, embedding API, LLM, account, or network request.
- **Tests:** `tests/lab02-mini-rag-studio.test.mjs` covers overlap, ranking, ties, citations, empty/invalid input, and deterministic output.
- **Limitations:** lexical retrieval misses semantic matches; extraction cannot synthesize new facts and is not a production RAG evaluation.
- **Route/files:** `#/lab/mini-rag-studio`; `miniRagStudio.js`; `MiniRagStudio.jsx`.

Operate it and reproduce the example

Select **Load example** to populate and immediately retrieve from the following fictional source:

The City Library opens at 9:00 AM from Monday to Saturday. It closes at 7:00 PM on weekdays and at 5:00 PM on Saturday. The library is closed on Sunday and public holidays.

Members may borrow up to five books for 21 days. A book can be renewed twice when no other member has reserved it. Reference books and newspapers must remain inside the library.

Membership is free for city residents who provide a photo ID and proof of address. Non-residents pay an annual fee of 500 rupees. Lost cards can be replaced at the service desk for 100 rupees.

Use the exact question **How long can a member keep a book, and when can it be renewed?**. With the UI defaults, the domain evaluation returns three indexed passages, two retrieved passages, and:

A book can be renewed twice when no other member has reserved it. [1] Members may borrow up to five books for 21 days. [2]

Citation [1] refers to the first ranked evidence item, source passage 2; [2] refers to source passage 1. Citation labels are evidence-list positions, not source passage IDs. Inspect **Top evidence** and its matched terms before using an answer. Edit source/question, then press **Retrieve answer**; edits do not recompute automatically. **Reset** clears both fields and output. This UI has no document upload, persistent index, or dedicated answer export control.

How retrieval and extraction work

- 1 Validate trimmed source length ≥ 40 characters and question length ≥ 6 . Invalid input returns empty chunks/evidence/citations and an empty answer. No maximum corpus size is enforced in the current validator: keep examples small because processing is synchronous.
- 2 Split the corpus on whitespace into default 48-word chunks with a 10-word overlap (stride 38). The example ranges are words 1-48, 39-86, and 77-100.
- 3 Lowercase and tokenize Unicode letters/numbers. Remove the defined stop words and one-character query terms; deduplicate query terms. A simple normalization removes a trailing s from terms longer than three characters unless they end in ss. It is not a general stemmer: `renew` and `renewed` remain different.
- 4 Score each passage as $\text{matched distinct query terms} / \text{distinct query terms} + \min(\text{matched-term occurrence count} / \text{passage token count}, 0.1)$. Keep passages with at least one match, sort descending score, then ascending passage ID for ties; return up to three by default.

- 5 Split evidence into punctuation-delimited sentence candidates. Rank by number of matching query terms, then evidence rank, then sentence order; remove exact duplicate sentences and choose at most two. Append citations to the chosen sentences. No matching sentence returns `No grounded answer was found in the supplied text`.

For the example, ranked passage IDs are 2 then 1, raw scores are 0.7 and approximately 0.678431; the UI displays scores multiplied by 100 as **70.0** and **67.8**. These are lexical relevance scores, **not confidence probabilities**. Overlapping chunks can begin/end inside a sentence, so extraction may produce a partial sentence in other examples.

Build map tests and troubleshooting

Implement and test `tokenize/chunkCorpus`, then `retrieveEvidence`, then `composeExtractiveAnswer`, then the `runMiniRag` orchestrator in `miniRagStudio.js`. `MiniRagStudio.jsx` owns `form/result` state and renders evidence; it performs no network call. UI defaults differ from the test that explicitly passes 35-word chunks and overlap 8, so do not copy that test's chunk count into the UI walkthrough.

Run `node --test tests/lab02-mini-rag-studio.test.mjs`: four tests passed in the 2026-08-31 documentation subbatch. They cover chunk overlap, ranked retrieval, the cited example, and invalid short input. If retrieval is empty, use words present in the source rather than assuming semantic similarity. For a slow page, reduce the corpus; no background worker or production-sized limit exists. Build interview: "I exposed retrieval and citations with deterministic lexical scoring; I did not connect a language model or claim semantic understanding." Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 03 GAN Latent Gallery

- **Certificate connection:** Build Basic Generative Adversarial Networks (GANs).
- **Real user/problem:** a designer or student needs a reusable abstract background asset and wants to explore nearby latent-style variations reproducibly.
- **Why built:** to provide a useful visual export while honestly demonstrating latent-coordinate ideas without pretending a trained GAN is present.
- **Inputs:** bounded X/Y latent coordinates, seed, style, and selected main/nearby variation.
- **Decision logic:** deterministically maps coordinates, seed, and style into an SVG composition and four nearby variations; selection fixes the artifact signature.
- **Outputs/artifacts:** preview gallery and downloadable 1600 × 1600 SVG with accessible title plus seed/style/coordinate/signature metadata and a useful filename.
- **Safety/privacy:** browser-local generation; no model download, training data, API, upload, or trained-GAN claim.
- **Tests:** `tests/lab03-gan-latent.test.mjs` covers determinism, coordinate sensitivity, bounds, selection/export stability, XML escaping, metadata, and filename generation.
- **Limitations:** this is a deterministic latent-space simulation, not adversarial training or learned image generation.
- **Route/files:** `#/lab/gan-latent-gallery`; `ganLatentModel.js`; `GanLatentGallery.jsx`.

Operate it and reproduce an export

- 1 Select **Load example**: X 2.4, Y -1.8, seed `deepan-lab-03`, style **Orbital bloom** (orbit). The main sample signature is `31e9cd42`, with nine generated shapes and four palette entries.
- 2 Select the main result or one of four nearby variations. The example variation coordinates/signatures are `(3.3, -1.8) / 2f38a760`, `(2.4, -0.9) / 52f9a626`, `(1.5, -1.8) / 3ff2064c`, and `(2.4, -2.7) / b00d332a`.
- 3 Select **Download selected SVG**. For the main sample the filename is `abstract-background-orbit-31e9cd42.svg`. The file has a 100×100 viewBox and 1600×1600 declared dimensions, accessible title/description, and seed/style/coordinate/signature metadata.
- 4 Move a slider or change the seed/style. The preview recomputes immediately and selection returns to the main sample; the previously downloaded file does not change.
- 5 **Reset** restores X/Y 0, seed `latent-demo`, style `orbit`. It does not produce an empty form.

Input rules and deterministic generation

Coordinates must be finite numbers in $-10 \dots 10$. UI sliders step by 0.1. Seed is trimmed, required, and at most 40 characters. Style must be `orbit`, `mosaic`, or `ripple`. An empty coordinate string passed directly to the domain function is converted by JavaScript `Number` to zero; the slider UI avoids that blank-value path. Invalid controls return `{valid:false, sample:null}` with errors and no exportable sample.

`hashString` is an unsigned 32-bit FNV-style hash using initial state 2166136261 and multiplier 16777619. A Mulberry32 pseudorandom sequence is seeded from `seed|style|x.toFixed(4)|y.toFixed(4)`. Base hue is rounded from $(\text{hash}(\text{seed}) \% 360 + (x + 10) \times 11 + (y + 10) \times 7) \% 360$; four derived HSL colors and nine shapes then consume the deterministic sequence. Shape centers are clamped to $6 \dots 94$ in the viewBox, widths/heights are generated from $20 \dots 72$, and opacity from $0.3 \dots 0.8$. The separate signature hashes `seed:x:y:style` and formats eight hexadecimal digits; it is a reproduction identifier, not a security hash.

Nearby variants use default offsets $(+0.9, 0)$, $(0, +0.9)$, $(-0.9, 0)$, $(0, -0.9)$, clamped to the coordinate bounds. Clamping can make a boundary variant coincide with the main coordinate. More importantly, coordinates seed a hash: nearby coordinates are **not guaranteed to create a smooth learned latent interpolation**. The generator creates deterministic procedural artwork, not output from adversarially trained weights.

Build and export architecture

Build validation, seeded generation, nearby variants, SVG serialization, and filename generation in `ganLatentModel.js`. Keep the exported sample object as the shared contract so preview and download refer to the same selected sample. `GanLatentGallery.jsx` derives main/variation samples with memoization, stores selection/export status, and renders SVG. Download constructs a Blob, creates a temporary object URL, clicks a download link, removes it, and revokes the URL. `serializeLatentSvg` escapes XML-sensitive title/seed/metadata text; do not bypass escaping when extending it.

Run `node --test tests/lab03-gan-latent.test.mjs`: seven tests passed in the 2026-08-31 documentation subbatch, covering repeatability, changed signatures, invalid controls, clamped variants, SVG metadata, XML escaping, and filename safety. A successful status message reflects the browser

download action, not a filesystem-level verification. If a download looks different, compare selected signature/style/coordinates, not only the visible seed. In an interview: “I built reproducible procedural assets and honest provenance around latent-coordinate exploration; I did not train a GAN.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 04 CNN Feature Explorer

- **Certificate connection:** Convolutional Neural Networks.
- **Real user/problem:** a learner needs to see exactly how a small image kernel turns pixels into a feature map.
- **Why built:** to replace black-box CNN language with inspectable multiplication, summation, and normalization.
- **Inputs:** editable 5×5 grayscale values or preset, plus edge, sharpen, or blur kernel.
- **Decision logic:** validates pixel/kernel ranges and performs one stride-1, no-padding 3×3 valid convolution, then normalizes the output for display.
- **Outputs/artifacts:** numeric 3×3 feature map and visual intensity map.
- **Safety/privacy:** entirely local; no image upload, trained model, classifier, inference claim, or API.
- **Tests:** `tests/lab04-cnn-convolution.test.mjs` covers edge and blur results, validation, output ranges, normalization, and flat-map handling.
- **Limitations:** one educational convolution pass cannot represent training, pooling, learned filters, multiple channels, or classification.
- **Route/files:** `#/lab/cnn-feature-explorer`; `cnnConvolution.js`; `CnnFeatureExplorer.jsx`.

Operate it and check the arithmetic

Choose **Diagonal** in Input preset and **Edge detection** in Kernel. This is the initial state. The exact input and expected raw output are:

Input:	Edge output:
255 20 0 0 0	1450 -410 -295
20 255 20 0 0	-410 1450 -410
0 20 255 20 0	-295 -410 1450
0 0 20 255 20	
0 0 0 20 255	

The first output cell is $8 \times 255 - (255 + 20 + 0 + 20 + 20 + 0 + 20 + 255) = 1450$. Negative output is valid: these are signed feature responses, not input pixels. Every edit recomputes immediately. Change Kernel to **Box blur** to inspect local averages, or **Sharpen** to preserve and emphasize the center. **Load example** reloads the diagonal pixels but **keeps the selected kernel**. **Reset** makes all pixels zero and also keeps the kernel; it does not return to the initial edge/diagonal state.

Algorithm and input rules

Input is exactly 5×5 finite numeric values from 0 through 255; blank strings and out-of-range values are rejected with row/column-specific errors. The UI suggests integer steps, but the domain validator also accepts finite decimals in range. Kernels are fixed:

Edge (divisor 1):	Sharpen (divisor 1):	Box blur (divisor 9):
-1 -1 -1	0 -1 0	1 1 1
-1 8 -1	-1 5 -1	1 1 1
-1 -1 -1	0 -1 0	1 1 1

For output row/column (r, c) , calculate $\text{sum}(\text{pixel}[r+i][c+j] \times \text{kernel}[i][j]) / \text{divisor}$ for $i, j = 0 \dots 2$, rounding the result to two decimals. Stride is one and there is no padding: $(5-3)+1 =$

3 outputs per axis. The code applies the kernel directly without flipping it (the convention commonly used in CNN layers); these particular kernels are symmetric, so flipping would not change their result.

Display normalization is separate from raw arithmetic: $(\text{value} - \text{min}) / (\text{max} - \text{min})$, rounded to four decimals. A flat map returns intensity 0.5 everywhere to avoid dividing by zero. Thus an all-zero input produces numeric zeros but a mid-intensity visual map; this is deliberate display behavior, not a nonzero convolution. Uniform pixels of 90 with Box blur produce nine raw outputs of 90.

Build map and verification

Implement grid-shape/range validation first in `cnnConvolution.js`, then nested window multiplication, then display normalization. Keep raw and normalized maps distinct so presentation does not change results. `CnnFeatureExplorer.jsx` stores pixels, kernel, and selected preset; it derives results with memoization. Editing any pixel labels the preset custom. No image upload, model training, or image classification is present.

Run `node --test tests/lab04-cnn-convolution.test.mjs`: four tests passed in the 2026-08-31 documentation subbatch for the exact edge matrix, blur divisor, validation, and normalization. If the example differs, check the retained kernel before suspecting arithmetic. An empty pixel intentionally withholds the feature map; enter a valid value or reload a preset. The result is an inspectable teaching aid, not evidence of CNN accuracy. Interview explanation: “I preserved signed kernel output and separated visualization normalization, making every multiplication auditable.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 05 Attention Text Explorer

- **Certificate connection:** Natural Language Processing with Attention Models.
- **Real user/problem:** a learner wants to see why one token receives more attention than another without opaque learned weights.
- **Why built:** to expose a scaled-dot-product-style calculation, positional influence, and softmax normalization step by step.
- **Inputs:** a bounded sequence of up to 12 tokens and a selected or entered focus token.
- **Decision logic:** derives transparent token features, calculates focus similarity plus positional bias, uses numerically stable softmax, and ranks normalized weights deterministically.
- **Outputs/artifacts:** score table, ranked weights, heat-style token sequence, and plain-language summary.
- **Safety/privacy:** local only; explicitly an attention visualization, not a transformer or language model.
- **Tests:** `tests/attention-text-explorer.test.mjs` covers tokenization, limits, repeatability, softmax stability/normalization, ranking, and a focus token outside the sequence.
- **Limitations:** transparent handcrafted features and position bias do not reproduce learned embeddings, heads, layers, or model behavior.
- **Route/files:** `#/lab/attention-text-explorer`; `attention-text-explorer.js`; `AttentionTextExplorer.jsx`.

Operate it with the exact sample

The initial example and **Load example** use sequence The curious robot studies language patterns carefully with focus language. Weights update as you type; there is no submit button. Open **Show the score calculation** to inspect features, similarity, distance, bias, raw score, and weight. The verified domain output has focus index 4 (zero-based), top token language, and these displayed percentages:

Token in source order	Weight shown to one decimal
The	12.3%
curious	14.2%
robot	13.6%
studies	14.8%
language	17.4%
patterns	14.2%
carefully	13.5%

Underlying weights sum to one; rounded labels need not sum to exactly 100.0%. Ranking is language, studies, curious, patterns, robot, carefully, The. Curious and patterns display the same rounded percentage but have different underlying weights.

Input and score specification

Whitespace splits display tokens. Accept 2-12 tokens, each at most 24 characters and containing at least one ASCII letter/digit after cleaning. Focus is required and at most 24 characters. Cleaning lowercases with the English locale and strips everything except a-z0-9; text made only from non-ASCII letters is rejected by this model even though the UI can display Unicode. Punctuation remains visible but does not contribute to features.

Each cleaned token becomes three features: length/24 (capped at one), number of aeiou vowels divided by cleaned length, and mean (characterCode % 31)/30. For language, the vector is approximately [0.333333, 0.5, 0.383333]. These are handcrafted features, not learned embeddings.

```

similarity[i] = dot(focusFeatures, tokenFeatures[i]) / sqrt(3)
distance[i]   = abs(i - focusIndex)
positionBias  = 0.35 / (distance[i] + 1)
rawScore[i]   = similarity[i] + positionBias
weight[i]     = exp(rawScore[i] - maxScore) / sum(exp(rawScore - maxScore))

```

The first cleaned match determines focus position. If focus is absent, position is $\text{floor}((\text{tokenCount}-1)/2)$, the left center in an even-length sequence. With alpha beta gamma delta and focus query, the anchor is index 1 (beta), not a fictitious new token. Rank by decreasing unrounded weight and increasing original index on exact ties. The example language self-similarity is approximately 0.293326; add 0.35 for raw score 0.643326, yielding weight approximately 0.174057 after normalization across all seven scores.

Build test and diagnose

Build tokenizer/validator, feature derivation, scaled dot product, stable softmax, and ordered result rows in `attention-text-explorer.js`. Subtracting the maximum before exponentiation improves numerical stability without changing the normalized proportions. `AttentionTextExplorer.jsx` derives tokens/result from sequence/focus state, supplies a datalist of sequence tokens, and displays heat strength relative to the largest weight; heat brightness is not itself a probability scale. **Reset** clears inputs and shows validation rather than computing an empty distribution.

Run `node --test tests/attention-text-explorer.test.mjs`: six tests passed in the 2026-08-31 documentation subbatch, covering whitespace, invalid limits/punctuation, feature repeatability, stable softmax, deterministic normalized ranking, and external-focus anchoring. If the “closest meaning” word does not win, inspect position/character features: the model does not understand meaning. No multi-head attention, learned query/key/value projections, value aggregation, or transformer inference is implemented. Interview explanation: “I exposed each score component and a stable softmax while explicitly separating this teaching simulation from a language model.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 06 Neural Network Playground

- **Certificate connection:** Neural Networks and Deep Learning.
- **Real user/problem:** a learner needs to trace how inputs, weights, and bias produce a neuron's output.
- **Why built:** to turn the neuron equation into visible arithmetic and small binary predictions.
- **Inputs:** two inputs, two weights, bias, and step, sigmoid, or ReLU activation.
- **Decision logic:** calculates each weighted contribution, sums the bias, applies the chosen activation, and repeats the configured neuron over four binary input pairs.
- **Outputs/artifacts:** equation, contribution breakdown, pre-activation value, activation result, and four sample predictions.
- **Safety/privacy:** deterministic and local; no training, model API, dataset, or performance claim.
- **Tests:** `tests/lab06-neural-network-playground.test.mjs` covers activations, weighted arithmetic, an OR-style configuration, and invalid numbers.
- **Limitations:** one manually configured neuron is not a multilayer network and does not learn.
- **Route/files:** `#/lab/neural-network-playground`; `neuralNetworkPlayground.js`; `NeuralNetworkPlayground.jsx`.

Operate the OR gate example

Select **Load OR-gate example**. Exact values are Input 1 1, Input 2 0, Weight 1 1, Weight 2 1, Bias -0.5, Activation **Step**. The result is:

```
(1 × 1) + (0 × 1) + -0.5 = 0.5
Contributions: 1, 0, -0.5
Weighted sum: 0.5
Output: 1.0000
Binary pairs 00, 01, 10, 11: outputs 0, 1, 1, 1
```

For a different experiment, enter all five numeric values, select an activation, then press **Run neuron**. Editing any field now clears the existing decision and binary table; run again to validate and display the new configuration. **Reset** clears both form and stored result. There is no training button or weight

optimization; users choose weights themselves. A one-neuron Step configuration can express this OR boundary but cannot express XOR in this two-input architecture.

Formulas validation and limits

$z = \text{input1} \times \text{weight1} + \text{input2} \times \text{weight2} + \text{bias}$. Step returns 1 when $z \geq 0$, otherwise 0: the zero boundary belongs to the positive class. ReLU returns $\max(0, z)$. Sigmoid returns $1/(1+\exp(-\text{clamp}(z, -500, 500)))$; the clamp bounds exponential computation rather than changing inputs in the displayed equation. For the example $z = 0.5$, sigmoid is approximately 0.622459 and ReLU is 0.5. All displayed contributions/output use four decimal places, while the internal calculation remains a JavaScript number.

Validation rejects empty strings, null/undefined, non-finite numeric inputs, and unknown activation names. It does not impose magnitude limits or check overflow of intermediate products: very large finite operands can still produce non-finite arithmetic. Use small educational values rather than treating this as a numerical computing package. Invalid domain input returns `output:null`, `weightedSum:null`, and an empty contribution array.

Build architecture and edit state correction

Build `validateNeuronInput`, `applyActivation`, and `runNeuron` in `neuralNetworkPlayground.js`; add `predictBinaryGrid` as four calls that substitute $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$ while retaining the selected weights/bias/activation. Keep domain tests independent of rendering. `NeuralNetworkPlayground.jsx` stores editable form separately from submitted result.

The audit found that the old component recomputed the binary table from an edited form while retaining the last submitted main result; clearing a weight could then format a null output with `toFixed`. The coordinator corrected `updateField` by clearing `result` on every edit. This local source change was inspected: both result sections are now withheld until the next valid run. The browser regression for editing/clearing after a successful example is still pending; source inspection is not a substitute for that acceptance check. Preserve this invalidation behavior when redesigning controls.

Run `node --test tests/lab06-neural-network-playground.test.mjs`. Four domain tests passed in the 2026-08-31 documentation subbatch: activation boundaries, contribution arithmetic, OR truth table, and invalid numbers. They do not exercise React edit-state handling. If output disappears after an edit, that is the new intentional behavior: finish valid inputs and press Run neuron. Verify this clear-on-edit path in the browser before accepting the redesigned page. Interview explanation: “I decomposed a neuron into weighted contributions and activation boundaries, without pretending a hand-set neuron is a trained network.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 07 GenAI Lifecycle Explorer

- **Certificate connection:** Introduction to Generative AI.
- **Real user/problem:** a project planner needs to see what evidence and controls are required between an idea and a monitored release.
- **Why built:** to challenge “prototype equals production” thinking with explicit gates and exit artifacts.
- **Inputs:** use case, data readiness, impact/risk choices, and deployment context.

- **Decision logic:** applies a transparent 0-100 readiness heuristic, detects blockers, and orders six delivery stages with required artifacts and governance actions.
- **Outputs/artifacts:** readiness band, blockers, six-stage lifecycle, exit evidence, and governance requirements.
- **Safety/privacy:** planning only; no dataset inspection, model call, provider approval, legal conclusion, or deployment decision.
- **Tests:** `tests/lab07-genai-lifecycle-explorer.test.mjs` covers stage order, high-impact controls, missing-data blockers, scoring, and validation.
- **Limitations:** the score is an educational heuristic and must not be used as production authorization.
- **Route/files:** `#!/lab/genai-lifecycle-explorer`; `GenAiLifecycleExplorer.js`; `GenAiLifecycleExplorer.jsx`.

Operate it with a reproducible plan

- 1 Select **Load example**. Use case is Help support agents draft answers from an approved internal product knowledge base. Data readiness is `partial`, risk impact `medium`, deployment context `internal`.
- 2 Expected score is $30 + 24 + 10 + 17 = 81/100$, labelled **Ready for a controlled prototype**, with no rule-generated blockers. This label is not a launch approval.
- 3 Read the fixed six-stage order: Scope → Data → Evaluate → Govern → Deploy → Monitor. Each stage supplies a decision and an exit artifact; the output does not actually create a dataset, run an evaluation, deploy a system, or start monitoring.
- 4 The example artifact list has six entries: Use-case charter; Success and failure metrics; Test set; Risk register; Deployment runbook; Monitoring log.
- 5 Change selections and press **Build lifecycle plan** to rebuild. Form edits alone leave the old plan displayed. **Reset** clears form and result. There is no save/export/copy button in this component.

Exact scoring and rule table

Factor	Choice → points
Fixed baseline	30
Data readiness	none → 0; exploring → 12; partial → 24; ready → 35
Risk impact	low → 18; medium → 10; high → 0
Deployment context	internal → 17; customer → 9; automated → 0

Sum the four terms. Scores ≥ 80 get Ready for a controlled prototype; 60-79 get Prepare key controls before prototyping; below 60 get Resolve blockers before building. The possible score range is 30-100, despite the `/100` display.

Blockers are determined separately: `none` data warns no usable dataset; `exploring` warns unproven data quality/permission/representation; `high` impact requires specialist review, appeal, and a non-AI fallback; `automated` requires an owner, stop control, and approval policy. High impact adds Impact assessment and Appeal and redress procedure. Customer-facing context adds User disclosure and feedback design plus Abuse test report. Automated context adds Human-override procedure plus Rollback and kill-switch test. Artifact deduplication preserves first occurrence.

A verified edge case is `ready + high + automated`: score 65, **two blockers**, and ten artifacts. Another is `none + low + internal`: score 65 with a no-dataset blocker. Read blocker text even when the numeric band sounds promising; the heuristic is not a decision authority and does not override missing evidence.

Build validate and troubleshoot

Use-case text needs at least 20 trimmed characters; select one key from each fixed option map. There is no upper use-case length cap in this validator. Missing/unsupported choices produce keyed errors, null score, and empty stage/artifact/blocker arrays. A false sense of “no blockers” from an empty invalid result is prevented by the UI's validation rendering.

Reconstruct the three point maps first, then validate keys/text, then calculate score and band, then add independent blockers/artifacts, and finally produce ordered stage objects. Keep these steps in `genAiLifecycleExplorer.js`; let `GenAiLifecycleExplorer.jsx` own only form/result state and accessible choice controls. Keeping blocker rules independent from scoring makes it possible to inspect why a concern exists without reverse-engineering a single opaque number.

Run node `--test tests/lab07-genai-lifecycle-explorer.test.mjs`: four tests passed in the 2026-08-31 documentation subbatch for the 81-point example, high-impact automated controls, missing-data blocker, and invalid inputs. If a plan stays unchanged, submit it again. If a high score has blockers, resolve those evidence gaps rather than adjusting ratings to remove warnings. Interview explanation: “I modeled lifecycle gates and separate safety blockers, using explicit heuristics rather than claiming to certify readiness.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 08 AI Opportunity Scorer

- **Certificate connection:** AI for Everyone.
- **Real user/problem:** a business planner needs an explainable first-pass screen for whether a repetitive task is a suitable AI candidate.
- **Why built:** to keep data readiness, error cost, privacy, and human judgment beside automation potential.
- **Inputs:** bounded 1-5 ratings for frequency, repetition, data readiness, error impact, privacy sensitivity, and judgment intensity.
- **Decision logic:** applies a visible weighted formula to a 100-point scale, maps the result to a recommendation band, and adds risk-specific guardrails.
- **Outputs/artifacts:** score, band, contribution breakdown, rationale, and next-step guardrails.
- **Safety/privacy:** local; no business data upload, ROI forecast, investment recommendation, or implementation approval.
- **Tests:** `tests/lab08-ai-opportunity-scorer.test.mjs` covers the example, inverse scenarios, high-risk guardrails, bounds, and deterministic scoring.
- **Limitations:** supplied ratings are subjective and the score cannot prove value, feasibility, permission, cost, or safety.
- **Route/files:** `#/lab/ai-opportunity-scorer`; `aiOpportunityScorer.js`; `AiOpportunityScorer.jsx`.

Operate it and reproduce the score

Select **Load example** for Classify incoming customer-support tickets. The ratings and points are:

Factor	Rating	Weight	Direction	Points
Task frequency	5	20	Positive	20
Process repetition	5	20	Positive	20
Data availability	4	20	Positive	15
Value of reducing errors	3	10	Positive	5
Privacy and sensitivity	2	15	Inverse	11.25
Human judgment required	2	15	Inverse	11.25

Points sum to 82.5; the final integer display uses `Math.round`, producing **83/100 - Strong automation opportunity**. The example guardrail is **Monitor quality and route low-confidence cases to a person**. The tool does not supply actual confidence estimates; this is planning advice. Results update as sliders/task text change. **Reset** restores blank task name and all six ratings to 3, which shows the required-task validation until a name is entered.

Exact rules and interpretation

Positive-factor points are $(\text{rating}-1)/4 \times \text{weight}$; inverse-factor points are $(5-\text{rating})/4 \times \text{weight}$. Each factor is rounded to two decimals, then the summed score is rounded to an integer before selecting its band. Bands are ≥ 75 Strong automation opportunity; 55-74 Promising pilot opportunity; 35-54 Use AI as an assistant; below 35 Low automation fit.

The `errorCost` key is presented as **Value of reducing errors**, and higher ratings add opportunity points. Separately, ratings ≥ 4 for this factor add a staged-rollout/audit/fallback warning because mistakes are costly. Do not mistakenly describe this factor as an inverse safety penalty. Privacy ≥ 4 adds privacy review; human judgment ≥ 4 adds human approval; data availability ≤ 2 adds data-quality/permission work; repetition ≤ 2 adds workflow standardization. If no specific warning triggers, the generic monitoring/human-review guardrail is used. These warnings do not numerically cap or override the band.

Task name is trimmed, required, and at most 120 characters. Every rating must be an integer from 1 to 5. Unsupported values return invalid status, a null score/band, and empty breakdown/guardrail arrays. The UI slider constrains input but the domain function independently checks it.

Build walkthrough and evidence

Create the factor metadata table first so labels, weights, directions, and rendering share one definition. In `aiOpportunityScorer.js`, add validation, per-factor normalization, final rounding/bands, and separate guardrail rules. `AiOpportunityScorer.jsx` derives the assessment from current input with `useMemo`; there is no submit state, network request, storage, or artifact-export function. The breakdown table is the main audit artifact and can support a planning discussion, not a forecast of savings or ROI.

Run `node --test tests/lab08-ai-opportunity-scorer.test.mjs`: four tests passed in the 2026-08-31 documentation subbatch. They verify the 83-point example and exact breakdown, the

30-point change across both inverse factors, high-risk guardrails, and invalid task/rating errors. If the total differs from adding displayed points, check final rounding. If a “strong” result still has warnings, the score and guardrails intentionally answer different questions. Interview explanation: “I made the screening formula and its subjective assumptions visible, with independent risk guidance instead of an automated investment recommendation.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 09 Prompt Workbench

- **Certificate connection:** Prompt Engineering for ChatGPT.
- **Real user/problem:** a prompt author needs a structured, reusable prompt and a visible completeness check before using it elsewhere.
- **Why built:** to demonstrate role/task/context/constraints/example/format composition without pretending to judge a model response.
- **Inputs:** role, task, context, constraints, example, and output format.
- **Decision logic:** assembles the sections, applies six deterministic quality checks, calculates a local structure score/band, and suggests missing details.
- **Outputs/artifacts:** copy-ready prompt, check results, score/band, and improvement suggestions.
- **Safety/privacy:** no model call; entered prompt text remains local and should not contain secrets.
- **Tests:** `tests/lab09-prompt-workbench.test.mjs` covers deterministic assembly, strong and incomplete examples, improvements, empty input, and validation.
- **Limitations:** structural checks do not measure correctness, safety, or response quality from any specific model.
- **Route/files:** `#/lab/prompt-workbench`; `promptWorkbench.js`; `PromptWorkbench.jsx`.

Operate it and inspect the assembled artifact

Select **Load example**. It uses the following six exact field values:

```
Role: You are a careful junior software-engineering mentor.
Task: Review the supplied JavaScript function and identify the three highest-impact improvements.
Context: The audience is a first-year computer-science student preparing an internship portfolio.
Constraints (three lines):
Do not rewrite the whole function.
Explain each recommendation in plain language.
Do not invent missing requirements.
Reference example: Finding: Unvalidated numeric input. Improvement: Reject non-finite values before calculation.
Output format: Return a numbered list with: finding, why it matters, and a concise suggested change.
```

The expected result is **100/100 - Strong**, all six checks Pass, no suggestions. The assembled prompt contains sections in this exact order: **## Role, ## Task, ## Context, ## Constraints, ## Reference example, ## Output format**. Constraint lines are trimmed, existing leading -, *, or • bullets are removed, empty lines are discarded, and each remaining line gets a single - prefix. Other field text is trimmed and preserved.

Edit fields to see assembly/checks update immediately. Select **Copy prompt** and look for **Prompt copied..** If clipboard access is unavailable or fails, the UI explains the failure and lets you select the read-only prompt text and copy manually; focusing that output selects its contents. **Reset** clears all fields

and copy status. This action assembles a prompt; it does not submit the prompt to any model or evaluate a response.

Validation versus scoring

All six fields are required after trimming. Maximum lengths are Role 300, Task 800, Context 1200, Constraints 1200, Reference example 1200, Output format 600 characters. The UI does not limit typing at these lengths; domain validation withholds the assembled prompt when a limit is exceeded.

Check	Rule	Points
Role specific	At least 25 trimmed characters	15
Task actionable	At least 40 characters and a supported action word	20
Context sufficient	At least 50 characters	15
Constraints separated	At least two nonempty normalized lines	15
Reference useful	At least 35 characters	15
Format explicit	At least 30 characters and a supported structure word	20

Task action words are create, write, review, analyze/analyse, compare, identify, design, explain, summarize/summarise, classify, calculate, build, and return. Format words are list, table, json, markdown, paragraph, section, field, column, bullet, numbered. Checks are case-insensitive whole-word regular expressions. Band thresholds are ≥ 85 Strong; ≥ 65 Good foundation; ≥ 40 Needs detail; otherwise Incomplete.

Validation and quality checks are independent. Six short nonempty values (Helper, Do this, Small context, Be clear, Short, Answer) are valid for assembly but score zero and produce six improvement suggestions. Conversely, long text may pass scoring checks while failing a maximum-length rule; a high score does not make an invalid prompt exportable. The word “quality” in the UI means these six structural heuristics only, not factuality, safety, or actual model-response quality.

Build test and troubleshoot

Build field/length validation and constraint normalization in `promptWorkbench.js`, then deterministic assembly, then six weighted check records and suggestions. `analyzePrompt` combines assembly validity with the separate score. `PromptWorkbench.jsx` keeps input and copy status, memoizes analysis, and renders a read-only artifact only when valid. Clear status on edits so “copied” never refers ambiguously to newly changed text.

Run `node --test tests/lab09-prompt-workbench.test.mjs`: four tests passed in the 2026-08-31 documentation subbatch for deterministic headings/bullets, full example score, specific low-detail suggestions, and all required-field errors. If a semantically good task fails a check, inspect the supported words/length threshold; the rule does not understand intent. If copy fails, use manual selection rather than entering credentials or changing browser security settings. Interview explanation: “I built a reusable local prompt artifact and transparent structural checks, keeping heuristic scoring separate from validity and model quality.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 10 ML Model Lab

- **Certificate connection:** Machine Learning Specialization.
- **Real user/problem:** a student or analyst needs to understand a complete small classification decision, including scaling, neighbours, votes, and evaluation.
- **Why built:** to make K-nearest-neighbours inspectable rather than returning only a label.
- **Inputs:** bounded CSV-style rows with two numeric features and a label, k, and query X/Y values.
- **Decision logic:** parses and validates rows, standardizes both features, computes Euclidean distances, selects neighbours, uses deterministic vote/tie rules, and performs leave-one-out evaluation.
- **Outputs/artifacts:** predicted label, nearest-neighbour table, vote evidence, standardization details, leave-one-out accuracy, and confusion summary.
- **Safety/privacy:** local data only; no upload, training service, API, or production-quality claim.
- **Tests:** tests/lab10-ml-model-lab.test.mjs covers parsing/errors, scaling, neighbours, deterministic ties, evaluation, and invalid k.
- **Limitations:** exactly two numeric features, bounded simple CSV, and KNN classification only; leave-one-out accuracy is a learning diagnostic, not production validation.
- **Route/files:** #/lab/ml-model-lab; ml-model-lab.js; MlModelLab.jsx.

Operate it with the exact eight row sample

The initial state and **Load example** use:

```
study_hours,practice_score,outcome
2,34,Developing
3,43,Developing
4,51,Developing
5,61,Developing
6,70,Ready
7,76,Ready
8,84,Ready
9,91,Ready
```

Query: study_hours 6.5, practice_score 74, k = 3. Results update as you type. Prediction is **Ready**, with the following nearest neighbours (distance displayed to three decimals):

Rank	study_hours	practice_score	Class	Standardized distance
1	7	76	Ready	0.243
2	6	70	Ready	0.305
3	8	84	Ready	0.844

All three votes are Ready; their unrounded distance total is approximately 1.391054. The leave-one-out diagnostic correctly classifies **7 of 8 rows (87.5%)** at k=3: actual Developing has three Developing predictions and one Ready; actual Ready has zero Developing and four Ready. These fictional labels do not justify assessing a real student's readiness.

Open **How distance is calculated** to inspect feature means/deviations. Change k to compare vote/evaluation changes. **Reset** clears dataset and query fields, resets k to 3, and displays validation. There is no training service, dataset upload, persistent model, export, or prediction API.

Parsing and accepted data

Use one three-cell header followed by 3-40 nonempty data rows, exactly two numeric features and one label. Header names must be nonempty and unique ignoring case. Quoted cells are rejected anywhere; this is a simple comma-split format, not a full CSV implementation. Use 2-8 distinct, case-sensitive class labels, each at most 32 characters. Query values must be nonblank finite numbers. k must be an integer from 1 to row count.

Important source caveat: dataset feature cells are converted with `Number(cell)` without first rejecting a blank trimmed cell, so an empty feature cell currently becomes zero. Explicitly fill every numeric feature; do not rely on the parser to flag missingness. There are no numeric magnitude caps, so extremely large finite inputs may overflow statistics/distances. These are limitations of the current educational parser, not accepted real-data cleaning behavior.

Exact distance tie and evaluation rules

For each feature, compute the training mean and population standard deviation: $\text{mean} = \text{sum}(v)/n$, $\text{sd} = \sqrt{\text{sum}((v-\text{mean})^2)/n}$. Use sd as scale, or 1 if sd is exactly zero. Distance is $\text{hypot}((\text{row}.x - \text{query}.x)/\text{scale}.x, (\text{row}.y - \text{query}.y)/\text{scale}.y)$; subtracting the same mean from both points cancels algebraically. Sample means are 5.5 and 63.75, deviations approximately 2.291288 and 18.799934. For (7, 76), the distance is $\text{hypot}(0.5/2.291288, 2/18.799934) \approx 0.242768$.

Sort neighbours by increasing distance, then original source-row index for equal distances. Give each of the first k rows one vote, without distance weighting. Sort class votes by decreasing vote count, then lowest sum of member distances, then `label.localeCompare` for remaining ties. With ordinary ASCII labels this matches the UI's alphabetical description; ordering for non-ASCII labels can depend on runtime collation. The selected prediction is the first vote entry, not a probability estimate.

Leave-one-out removes each row in turn, **recomputes scaling from the remaining training rows**, classifies the held-out point, and counts correct labels. Evaluation k is $\min(\text{requested } k, n-1)$; prediction with all n rows still uses requested k . That is why choosing k equal to n makes the displayed evaluation k one smaller. A constant feature has no variation between training rows; a query outside its constant value can still add a common nonzero distance term, so it is not universally “ignored.”

Build map test evidence and troubleshooting

Implement `parseKnnDataset` and `parseFiniteInput`, then feature statistics/distance, deterministic `classifyKnn`, and finally `calculateLeaveOneOut` in `ml-model-lab.js`. Derive confusion counts from per-row predictions rather than independently approximating accuracy. `MlModelLab.jsx` wraps parsing/classification/evaluation in `getAnalysis`, catches an error into one visible validation message, and memoizes the full analysis from dataset/query/ k state. With n rows, sorting for each leave-one-out iteration is roughly $O(n^2 \log n)$; the 40-row cap helps keep synchronous calculations small.

Run `node --test tests/lab10-ml-model-lab.test.mjs`: seven tests passed in the 2026-08-31 documentation subbatch for parsing/validation, unit scaling, neighbour voting, distance/alphabetic ties, evaluation/confusion totals, and evaluation- k capping. These tests use a different six-row fixture; the eight-row walkthrough above was separately evaluated directly from the domain module. If no result appears, inspect the first error, number of rows/classes, quoted values, and k . If results surprise you,

inspect standardized distances and ties before changing labels. Interview explanation: “I made preprocessing, votes, tie rules, and holdout scaling inspectable; accuracy here is a small-sample learning diagnostic, not production validation.” Final-UI screenshot: **capture pending after redesign acceptance**.

9 Labs 11 14 and 26 cloud and networking

Lab 11 Cloud Regret Pre Mortem

- **Certificate connection:** Introduction to Cloud Computing.
- **Real user/problem:** a founder or technical lead is considering a cloud migration and needs to expose the ways the decision could become regrettable before committing.
- **Why built:** to turn vague cloud enthusiasm or fear into comparable assumptions, failure scenarios, early warnings, and a reversible pilot.
- **Inputs:** decision context, current/planned cost assumptions, a bounded 12-36 month horizon, and five explicit risk ratings.
- **Decision logic:** compares stay/migrate planning totals from supplied assumptions, constructs exactly five regret scenarios, derives early-warning evidence, and gates a limited pilot with an exit test.
- **Outputs/artifacts:** comparison, five-regret pre-mortem, early warnings, reversible actions, 30-day pilot contract, exit test, and copyable decision memo.
- **Safety/privacy:** browser-local planning; no cloud account, provider API, live quote, pricing promise, migration instruction, or approval.
- **Tests:** `tests/lab11-cloud-regret-premortem.test.mjs` covers bounded inputs, scenario count/content, comparison math, warnings, pilot/exit controls, memo, and invalid input.
- **Limitations:** supplied costs and ratings are assumptions; the tool omits detailed architecture, contracts, taxes, service limits, and organization-specific risk.
- **Route/files:** `#/lab/cloud-regret-premortem`; `cloudRegretPremortem.js`; `CloudRegretPremortem.jsx`.

Worked example and exact result

Load the example: migrate the customer booking application and database; INR; current monthly cost 85,000; planned cloud monthly cost 68,000; migration cost 240,000; horizon 24 months; portability medium, lock-in medium, skills high, downtime medium, and export readiness low. The stay total is $85,000 \times 24 = ₹2,040,000$; the migrate total is $68,000 \times 24 + 240,000 = ₹1,872,000$; the planning difference is a ₹168,000 saving. Risk labels map low/medium/high to 1/2/3, so the five ratings total 10/15 with one high rating. The engine creates exactly five regret exposures: cloud bill ₹240,000, migration delay ₹180,000, outage ₹29,750, lock-in ₹320,000, and export/recovery ₹80,000. These are deterministic planning exposures, not forecasts.

Algorithm operation and acceptance boundary

Validation requires a decision goal of at least 20 characters, INR/USD/EUR/GBP, costs from 0 through 100,000,000, an integer horizon from 12 through 36, and all five supported ratings. $stayTco = currentMonthly \times horizon$; $migrateTco = cloudMonthly \times horizon + migrationCost$; $difference = stayTco - migrateTco$. The recommendation permits only a bounded pilot when $difference > 0$ and no more than one rating is high; otherwise it pauses full migration. The example pilot is one noncritical workflow with representative data for 30 days, while keeping the source path intact. Continue only when run-rate is no more than ₹74,800/month (110% of the supplied cloud estimate), export/restore succeeds, rollback remains possible inside the window, and the outcome is evaluated.

Use **Load example**, review every assumption, then **Run pre-mortem**. Read the comparison before the scenario cards; assign an owner to each early warning and reversible action; copy the decision memo only after agreeing the pilot exit test. **Reset** removes the scenario result. The UI never initiates a migration.

Build map tests and troubleshooting

`cloudRegretPremortem.js` owns validation, rating conversion, arithmetic, five-scenario construction, pilot gates, and memo formatting. `CloudRegretPremortem.jsx` owns controlled fields, submit/load/reset, result cards, and clipboard feedback. The focused test file has three tests: exact five-card/example math, pause behavior for negative economics or high risk, and cost/horizon bounds. If output is withheld, check the first field error, all five ratings, and integer horizon. If currency totals look surprising, remember that migration cost is added once while monthly costs are multiplied by the full horizon. Interview explanation: “I made a cloud recommendation conditional on both stated economics and reversibility; it is a pre-mortem, not a cost oracle.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 12 AWS Customer Journey Outage Storyboard

- **Certificate connection:** AWS Cloud Practitioner preparation.
- **Real user/problem:** a small launch team needs a safe GameDay discussion tied to what customers experience, not an unbounded infrastructure failure exercise.
- **Why built:** to connect journey steps and dependency categories to evidence, containment, fallback, recovery proof, and facilitation.
- **Inputs:** customer journey steps, dependency categories, recovery priority, traffic context, and data criticality.
- **Decision logic:** selects three to five journey-relevant failure cards and fills each with symptom, evidence, safe containment, fallback, recovery validation, and a stable AWS concept category.
- **Outputs/artifacts:** outage storyboard/GameDay cards and facilitator checklist.
- **Safety/privacy:** no AWS account, credentials, fault injection, provider call, production command, or promise that the cards cover every failure.
- **Tests:** `tests/lab12-aws-outage-storyboard.test.mjs` covers bounded journey parsing, card selection/count, journey linkage, recovery evidence, checklist output, and validation.
- **Limitations:** dependency categories are planning abstractions; teams must adapt cards to the real architecture and run only authorized non-production exercises.

- **Route/files:** `#/lab/aws-outage-storyboard; awsOutageStoryboard.js; AwsCustomerJourneyOutageStoryboard.jsx.`

Worked example and deterministic card mapping

The built-in journey is: Open product page; Sign in; Add item to cart; Pay for order; See order confirmation. Its dependencies are edge, compute, data, asynchronous work, and external provider; priority is protect data, traffic is launch spike, and data criticality is high. The five selected cards map in order to those five journey steps: customer cannot reach the experience; API path is unavailable; source of truth is unavailable or inconsistent; background work silently stops; and a third-party dependency degrades. Every card contains a customer symptom, two evidence checks, safe containment, fallback, two recovery proofs, and one stable AWS concept category. The example facilitator checklist contains ten items, including reconciliation of data after recovery and load appropriate to the launch.

Selection algorithm and safe use

Validation accepts two to eight nonblank journey steps of at most 100 characters, at least one supported dependency, and enumerated priority, traffic, and criticality values. Dependencies are deduplicated. Candidate failures are created from the chosen dependencies, then traffic and change scenarios are available to fill the bounded set. Card count is $\min(5, \max(3, \text{selectedDependencyCount}))$; therefore even one selected dependency yields a three-card rehearsal. For card index i , journey position is $\text{round}(i \times (\text{journeySteps}-1)/(\text{cardCount}-1))$, distributing the cards from the first to last customer step. This deterministic mapping explains the output and avoids pretending to discover the real architecture.

Use **Load example** or enter the customer-visible steps in order. Change context selectors and inspect cards immediately-the memo is derived from current input rather than produced by a destructive action. During a GameDay, read symptom first, nominate observers for the evidence, rehearse containment/fallback only in an authorized environment, and require both recovery proofs. **Reset** restores the blank planning state.

Build map tests and troubleshooting

`awsOutageStoryboard.js` contains parsing, category templates, sampling, card assembly, and checklist rules. The React screen keeps form state and computes the storyboard with `useMemo`; it never contacts AWS. Four focused tests cover the exact five journey-linked cards, guaranteed minimum three cards, priority/traffic variation, and all validation boundaries. If a card seems generic, replace a dependency category with the closest real dependency and rewrite the journey as customer actions; do not treat AWS concept labels as inventory discovery. Interview explanation: "I connected operational failure rehearsal to customer steps and recovery evidence without injecting faults or requiring credentials." Final-UI screenshot: **capture pending after redesign acceptance.**

Lab 13 Azure Access Handoff Simulator

- **Certificate connection:** Microsoft Azure Fundamentals (AZ-900).
- **Real user/problem:** a manager or Azure administrator needs to review joiner, mover, and leaver access handoffs without exposing a tenant or real identities.
- **Why built:** to convert a small roster into owned evidence actions instead of a generic identity checklist.

- **Inputs:** bounded local roster metadata such as status, job/role, resource, privilege, shared-credential indicator, and days since review.
- **Decision logic:** detects orphaned access, role/resource mismatch, excess scope, shared credentials, and review drift, then prioritizes an owner/evidence action.
- **Outputs/artifacts:** issue summary, ordered remediation actions, evidence requirements, and copyable handoff checklist.
- **Safety/privacy:** generic local roster only; no tenant connection, identity, email, credential, permission change, or compliance claim.
- **Tests:** `tests/lab13-azure-access-handoff.test.mjs` covers joiner/mover/leaver findings, shared/excess access, stale review, action ordering, validation, and checklist content.
- **Limitations:** cannot inspect effective permissions, group inheritance, conditional access, logs, or actual completion.
- **Route/files:** `#/lab/azure-access-handoff`; `azureAccessHandoff.js`; `AzureAccessHandoffSimulator.jsx`.

Worked example and exact findings

The example roster has four pipe-delimited rows: Anika/new/frontend developer/resource-group web-demo/contributor/no/0; Bala/active/support analyst/subscription startup-prod/reader/yes/120; Chen/role-change/data analyst/resource-group analytics/owner/no/95; and Divya/exiting/operations lead/subscription startup-prod/owner/yes/180. It validates four records and emits nine findings: Bala has a critical shared credential, medium excessive subscription scope, and medium review drift; Chen has high owner scope and medium review drift; Divya has critical orphan access, critical shared credential, high owner subscription scope, and medium review drift. The resulting 11 actions are sorted by priority, then person, then action. A clean active record still receives one “reconfirm attestation” action, so absence of a risk rule does not mean absence of review evidence.

Decision rules user workflow and evidence

The parser turns each local row into normalized lifecycle/access metadata and rejects malformed structures or unsupported values before planning. Findings cover four implemented risk families-shared credential, excessive scope, review drift, and orphan access-with severity-specific remediation. Plan construction pairs each finding with an owner, action, and evidence request; deterministic ordering keeps critical handoffs visible. This is a simulated review of declared facts: it cannot infer role suitability from Azure itself.

Paste synthetic or de-identified roster rows, validate the row grammar shown beside the input, and run the handoff. First compare record and finding counts; then work through critical actions before high and medium; finally copy the eight-point checklist into the approved handoff process. Never paste passwords, access tokens, email addresses, or an exported production directory.

Build map tests and troubleshooting

azureAccessHandoff.js owns row parsing, lifecycle/access rules, ordered action construction, and checklist text. AzureAccessHandoffSimulator.jsx handles local entry, load/reset, findings/actions, and clipboard state. Four focused tests prove all four risk types, stable priority ordering, clean-record attestation, and structural/field errors. If counts differ, inspect status, scope/role text, shared yes/no, and numeric days since review in the rejected row before changing a rule. Interview explanation: “The tool converts a joiner/mover/leaver declaration into owned evidence tasks while making clear that effective permission inspection remains external.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 14 GCP Transformation Promise Ledger

- **Certificate connection:** Google Cloud Digital Leader preparation.
- **Real user/problem:** an executive sponsor or small-organization project lead needs to challenge a cloud-transformation promise before making a commitment.
- **Why built:** to make an executive claim falsifiable and evidence-gated, rather than producing a generic migration plan or service list.
- **Inputs:** promise statement, target metric, current baseline, claimed target, workload count, data sensitivity, downtime tolerance, dependency complexity, team skill, and modernization intent.
- **Decision logic:** translates the promise into a falsifiable hypothesis, records assumptions/risks, designs the smallest reversible pilot, and creates 30/60/90 evidence gates with stop/continue triggers.
- **Outputs/artifacts:** hypothesis, assumption ledger, evidence needed, pilot, evidence gates, risk owners, inventory checklist, triggers, and copyable sponsor memo.
- **Safety/privacy:** local planning; no GCP account, service deployment, pricing/current-capability claim, guaranteed calendar, or approval. Official GCP sources are used only for stable concepts/names.
- **Tests:** tests/lab14-gcp-promise-ledger.test.mjs covers validation, hypothesis construction, assumptions, reversible pilot, evidence gates, triggers, risks, memo, and source boundaries.
- **Limitations:** user-supplied baselines/targets are not verified; evidence gates are review points, not guaranteed delivery dates or transformation proof.
- **Route/files:** #/lab/gcp-promise-ledger; gcp-promise-ledger.js; GcpTransformationPromiseLedger.jsx.

Worked example and falsifiable output

Load the promise that a customer-portal migration will reduce release downtime without weakening controls, with metric “Release downtime in minutes,” baseline 45, target 15, 12 workloads, high data sensitivity, scheduled downtime, medium dependency complexity, basic team skill, and managed modernization. The hypothesis states that the metric must move from 45 to 15—a decrease of 66.7%—under the same definition and observation conditions without breaching data, downtime, dependency, or rollback constraints. The smallest reversible pilot is two representative workloads; the source path remains intact and must not be retired. The output supplies owned assumptions/risks, 30/60/90 evidence gates, continue/stop triggers, an evidence inventory, and a sponsor memo whose decision is only continue, revise, or stop the pilot.

Algorithm and evidence policy

Promise text is capped at 280 characters, metric at 80, and workload count at 500; baseline and target must be finite and different, and every context selection must be supported. Change is $\text{target} - \text{baseline}$; direction follows its sign, magnitude is absolute, and percent is $\text{abs}(\text{delta}/\text{baseline}) \times 100$ unless baseline is zero. Pilot sizing is deliberately bounded and risk-sensitive; it does not scale into a migration plan. High sensitivity, dependency complexity, and limited skill add corresponding owned assumptions and risks. Gates are evidence review horizons, not guaranteed completion dates. Source links in the app are limited to official Google Cloud documentation and support concepts, not current service or price claims.

Use **Load example**, replace the promise and metric with measurable wording, then generate the ledger. Freeze the metric definition before collecting baseline evidence. Review assumptions and risks with named owners, run only the proposed reversible pilot, and treat each horizon as a decision gate. Copy the memo after evidence inventory and stop triggers are accepted.

Build map tests and troubleshooting

`gcp-promise-ledger.js` contains bounded validation, signed change, pilot sizing, risk-sensitive assumptions, gates, triggers, references, and memo. The React module renders these artifacts and performs no deployment. Seven focused tests cover all validation, deterministic hypothesis, two-workload example pilot and one-workload high-risk case, risk-sensitive additions, evidence gates, sponsor decision limits, and official URL boundaries. If no ledger appears, shorten statements, make baseline and target different finite values, and select every risk context. Interview explanation: “I converted transformation marketing language into a falsifiable hypothesis and reversible evidence gates, explicitly limiting the sponsor decision.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 26 Network Change Rollback Composer

- **Certificate connection:** The Bits and Bytes of Computer Networking.
- **Real user/problem:** a small-business network administrator planning an IPv4 change needs obvious unsafe address plans blocked and a rollback-ready communication package before touching devices.
- **Why built:** to combine CIDR reasoning with change control; it is intentionally more than a subnet calculator.
- **Inputs:** change goal, downtime window, before inventory, and proposed inventory, with up to 20 named IPv4 network rows.
- **Decision logic:** normalizes `/1-/30` CIDRs, calculates usable capacity, detects overlap/reserved-range/capacity blockers, diffs inventories, and withholds execution steps when blocked.
- **Outputs/artifacts:** change findings, precheck, execution or safety interlock, verification, rollback, and stakeholder summary.
- **Safety/privacy:** platform-neutral and local; no device connection, live route/ACL inspection, credentials, vendor commands, or automatic change.
- **Tests:** `tests/lab26-network-change-rollback.test.mjs` covers normalization, a useful example, overlap/capacity blockers, reserved ranges, row limits, and withholding execution on failure.
- **Limitations:** IPv4 only; no IPv6, routing protocol, ACL, DHCP-server, topology, or vendor-specific validation. A network owner must review and rehearse the runbook.
- **Route/files:** `#/lab/network-change-rollback`; `networkChangeRollback.js`; `NetworkChangeRollbackComposer.jsx`.

Worked example and CIDR calculations

The example changes Office from /24 to /25 for 90 required hosts, adds VoIP /26 for 35, and leaves Servers unchanged. A /25 has $2^{(32-25)} - 2 = 126$ usable hosts; a /26 has 62. The plan therefore has two concrete modifications, no deterministic findings, and is not blocked. Each normalized network also reports first usable, last usable, and broadcast addresses, which feed prechecks and verification-not device commands.

Parser blockers and rollback interlock

Each inventory accepts at most 20 comma-delimited `name,cidr,required-hosts` rows, unique names, /1 through /30, and required hosts from 1 through 1,000,000. Host-form CIDRs are normalized to their network boundary. Usable capacity is $2^{(32-\text{prefix})} - 2$. The analyzer blocks internal overlaps, required hosts greater than usable hosts, and reuse of space belonging to a removed pre-change network until drainage is proved. It warns when $\text{ceil}(\text{requiredHosts} \times 1.2)$ exceeds capacity or an address was boundary-normalized. It rejects 0/8, loopback, link-local, multicast or higher ranges. If any blocker exists, execution steps are withheld while precheck, rollback, and stakeholder evidence remain visible.

Enter the goal/window and before/proposed inventories, then run analysis. Resolve every blocker before using the runbook. Independently confirm actual routing, DHCP, ACLs, topology, reservations, and backups; obtain change approval; rehearse rollback; and only then translate the platform-neutral sequence into vendor-reviewed commands. Copying the summary is communication, not authorization.

Build map tests and troubleshooting

`networkChangeRollback.js` owns IPv4-number conversion, CIDR normalization, overlap/capacity checks, inventory diff, interlock, runbook, and summary. The React screen owns inputs and copy feedback. Four focused tests cover normalization/boundaries, useful example changes and rollback, overlap/capacity blocking, and reserved/row-limit rejection. If an apparently valid subnet warns, compare the supplied host address with its normalized boundary; if execution is absent, search findings for `Blocker`. Interview explanation: “I combined subnet arithmetic with a fail-closed change-control package; the browser never touches network equipment.” Final-UI screenshot: **capture pending after redesign acceptance**.

10 Labs 15 20 front end programming and data

Lab 15 Responsive Constraint Handoff

- **Certificate connection:** Introduction to Front-End Development.
- **Real user/problem:** a designer and front-end developer need to agree how dense components reflow across viewports before implementation.
- **Why built:** to turn component and content constraints into a buildable handoff rather than another visual breakpoint toy.
- **Inputs:** screen task, viewport widths, component priority, minimum width, content type, visibility rule, and related constraints.
- **Decision logic:** checks per-viewport feasibility/collisions, chooses stack/reflow/hide behavior, preserves “never hide” items, and derives keyboard order and CSS Grid guidance.

- **Outputs/artifacts:** viewport decision table, visibility/reflow rules, keyboard sequence, CSS starter, and QA checklist.
- **Safety/privacy:** local design metadata; no user tracking, content upload, deployed DOM inspection, or accessibility certification.
- **Tests:** `tests/lab15-responsive-constraint-handoff.test.mjs` covers parsing, feasibility, reflow, required visibility, focus order, generated CSS, QA, and validation.
- **Limitations:** estimates do not replace testing real fonts, content, localization, touch targets, screen readers, zoom, or browser layout.
- **Route/files:** `#/lab/responsive-constraint-handoff`; `responsiveConstraintHandoff.js`; `ResponsiveConstraintHandoff.jsx`.

Worked example and layout result

The example task is appointment search, comparison, and confirmation at viewports 360, 768, 1024, 1440. It declares Primary nav/critical/280/navigation; Search filters/critical/320/input; Available times/high/360/content; Booking summary/high/280/content; Confirm booking/critical/180/action; and Promo banner/low/300/decorative. The available content width is viewport minus 32 pixels, and row packing uses a 16-pixel gap. The four layouts contain 5, 3, 2, and 2 rows respectively. At 360 only the low-priority promo is hidden; navigation, input, and action are never hidden. Keyboard focus order remains Primary nav → Search filters → Confirm booking at every width. The output also includes collision warnings, per-component decisions, a CSS Grid starter, and seven QA checks.

Packing rules workflow and boundaries

Enter at most eight unique integer viewport widths and 20 unique component rows. Each row uses `name|critical/high/medium/low|min-width|navigation/input/action/content/decorative`; invalid interactions, duplicate names, and nonpositive widths fail validation. Components are ordered by priority and declaration order. `packRows` keeps adding `minWidth + 16` while it fits. When a layout still cannot fit, only eligible low-priority content/decorative candidates are hidden; critical items and navigation/input/action interactions are invariant. The CSS is a starting contract, not proof of browser behavior.

Load or enter a real screen inventory, generate the handoff, and review the narrowest viewport first. Confirm every hide/reflow decision with design and product, then paste the CSS starter into a branch and execute the seven checks with real content, zoom, keyboard, and assistive technology. A collision warning should drive a design decision, not be silenced by changing the declared width.

Build map tests and troubleshooting

`responsiveConstraintHandoff.js` owns both parsers, never-hide invariant, row packing, decisions, focus order, checklist, and CSS. `ResponsiveConstraintHandoff.jsx` renders the controlled input and per-viewport artifacts. Three focused tests cover the example's reflow/focus sequence, too-wide collision warnings, and all parser limits. If row count differs, account for the 32-pixel page inset and 16-pixel inter-item gap. Interview explanation: "I made responsive behavior an auditable constraint handoff with keyboard invariants, while keeping browser/accessibility testing as an explicit acceptance step." Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 16 Exception First Python Automator

- **Certificate connection:** Hello Python.
- **Real user/problem:** an office worker or junior developer needs a reviewable batch-file automation starter that anticipates failure and recovery before changing files.
- **Why built:** to teach safe automation through dry-run, path bounds, duplicates, audit, backup, and undo rather than a happy-path script generator.
- **Inputs:** task type, bounded path/glob metadata, naming template, copy/move choice, duplicate strategy, and failure behavior.
- **Decision logic:** validates path and traversal rules, then generates a Python standard-library scaffold with dry run, capped work, checksum/audit behavior, backups, error handling, and undo record.
- **Outputs/artifacts:** copyable Python source and reviewer checklist.
- **Safety/privacy:** the browser does not read/write files or execute Python; traversal such as `..` is rejected and source is withheld on unsafe input.
- **Tests:** `tests/lab16-exception-first-python-automator.test.mjs` covers valid generation, dry-run/audit/undo features, duplicate modes, path rejection, bounds, and invalid input.
- **Limitations:** generated code is a reviewed starter, not a tested production job; filesystem permissions, races, links, encodings, and organization policy need real validation.
- **Route/files:** `#/lab/exception-first-python-automator`; `exceptionFirstPythonAutomator.js`; `ExceptionFirstPythonAutomator.jsx`.

Worked example and generated safety contract

The example archives weekly PDFs from `C:/Office/Incoming/**/* .pdf` to `C:/Office/Archive/{date}/{name}`, using move, continue after an item failure, and rename on duplicate. It generates a 112-line Python standard-library scaffold with `DRY_RUN = True`, `MAX_FILES = 1000`, a JSON audit record, and an undo action that moves the item back. It catches a defined exception tuple rather than every possible failure, validates paths before acting, and does not use `subprocess`, `eval`, or `exec`. The seven-item review checklist keeps dry-run review, backup/undo, permissions, bounds, and audit evidence visible.

Validation operation and generated code boundary

The form requires a meaningful task name, source pattern, destination template, and supported operation/failure/duplicate selections. It rejects `..` traversal, an unsafe destination wildcard, and placeholders other than `{date}`, `{name}`, `{stem}`, and `{suffix}`. The generator embeds choices deterministically, caps matched files at 1,000, resolves duplicates according to the selected policy, and emits audit/undo metadata. It intentionally does not inspect the filesystem or guarantee that Windows glob and permissions match the user's environment.

Load the example, edit the plan, and generate. Copy the source into a separate reviewed file; keep `DRY_RUN` true; use disposable fixtures; inspect every planned source/destination and audit record; test duplicate and failure behavior; verify undo; only then consider enabling mutation under normal backup and approval controls. The portfolio screen itself never runs the script.

Build map tests and troubleshooting

exceptionFirstPythonAutomator.js owns plan validation, Python literal escaping, policy-specific source generation, and checklist. The React screen owns fields, output visibility, and copy state. Four focused tests prove dry-run/audit/undo generation, deterministic policies, unsafe/unbounded rejection, and placeholder rejection. If source is withheld, remove traversal/wildcards and use only supported placeholders. If a copied script finds unexpected files, stop in dry-run and narrow the source glob; do not increase the cap first. Interview explanation: “The deliverable is an exception-first, dry-run scaffold with audit and undo evidence-not browser file automation.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 17 Queue Fairness Replay

- **Certificate connection:** Data Structures in C.
- **Real user/problem:** a helpdesk, clinic, or event manager and a C student need to compare fast urgent service with fairness to earlier arrivals.
- **Why built:** to make queue policy consequences visible and produce a concrete implementation contract.
- **Inputs:** bounded cases with arrival, urgency, and service duration plus an aging setting.
- **Decision logic:** simulates one non-preemptive server under FIFO and priority-with-aging, then calculates dequeue order, wait time, maximum wait, starvation signal, and fairness evidence.
- **Outputs/artifacts:** side-by-side replay, waits/fairness comparison, and copyable C data-structure/algorithm contract.
- **Safety/privacy:** generic cases only; no identities, live queue integration, dispatch action, or claim that the policy is ethically optimal.
- **Tests:** tests/lab17-queue-fairness-replay.test.mjs covers parsing, FIFO, aging priority, wait calculations, tie/order rules, starvation/fairness output, and C contract.
- **Limitations:** simplified single server, no preemption, skills, breaks, balking, abandonment, parallel workers, or service-level policy.
- **Route/files:** #/lab/queue-fairness-replay; queueFairnessReplay.js; QueueFairnessReplay.jsx.

Worked example and exact replay

The example cases are WALK-IN-01|0|2|8, URGENT-02|2|5|12, CALL-03|3|3|5, FOLLOWUP-04|5|1|4, and URGENT-05|9|4|7, with aging interval 10 minutes. FIFO follows input/arrival order and reports average wait 12.6, maximum wait 20, fairness 33/100, and zero starvation flags. Priority-aging orders WALK-IN-01, URGENT-02, URGENT-05, CALL-03, FOLLOWUP-04 and reports average wait 13.6, maximum wait 27, fairness 33/100, and zero starvation flags. This demonstrates that urgency can change order without automatically improving every wait measure.

Scheduler and fairness formulas

Rows require a unique 1-24 character ID using letters, numbers, underscore, or hyphen plus bounded numeric arrival/urgency/service values; the parser accepts at most 30 cases. The aging interval is validated separately. Both simulations use one non-preemptive server; when no case is available, time advances to the next arrival. FIFO chooses earliest arrival then input order. Priority-aging recalculates $\text{score} = \text{urgency} + \text{floor}((\text{currentMinute} - \text{arrivalMinute}) / \text{agingMinutes})$ at each dequeue; higher wins, ties use arrival then input order. A wait of at least 30 minutes is flagged as starvation. Fairness is Jain's index over promptness values $1 / (1 + \text{wait})$, multiplied by 100 and rounded; it describes distribution, not ethics.

Enter synthetic cases, run the replay, compare order and per-case waits before aggregate scores, and copy the bounded C contract only after choosing a policy with stakeholders. Test the contract against the exact emitted vector in a separate compiler. Do not paste patient/customer identities.

Build map tests and troubleshooting

`queueFairnessReplay.js` owns parsing, time simulation, tie rules, metrics, starvation, and C contract generation. The React screen visualizes the two schedules. Four focused tests cover exact deterministic orders, eventual aging promotion, parser/interval bounds, and copyable C vector. If aging appears ineffective, inspect when a case actually becomes available and remember scores update only at dequeue boundaries. Interview explanation: “I exposed the service-policy trade-off with reproducible tie rules and a transparent fairness proxy rather than claiming one queue is universally fair.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 18 Safe C Input Harness

- **Certificate connection:** Programming in C.
- **Real user/problem:** a C student or embedded junior needs bounded console-input code and failure tests before trusting user input.
- **Why built:** to make input validation and memory bounds the primary artifact, not an afterthought to a small program.
- **Inputs:** field definitions with supported type, numeric bounds, maximum text length, and required/optional choice.
- **Decision logic:** validates the field contract, generates `fgets` plus `strtoul/strtod` parsing patterns, creates boundary/invalid vectors, and estimates declared stack buffers.
- **Outputs/artifacts:** C input scaffold, negative test vectors, stack expectation, and reviewer checklist.
- **Safety/privacy:** generates text only; does not compile, execute, read devices/files, or accept production data.
- **Tests:** `tests/lab18-safe-c-input-harness.test.mjs` covers field parsing, integer/decimal/text generation, bounds, invalid vectors, stack estimate, and malformed definitions.
- **Limitations:** generated code needs compiler/platform review and testing for locale, encoding, integer width, I/O environment, fuzzing, and surrounding program logic.
- **Route/files:** `#/lab/safe-c-input-harness`; `safeCInputHarness.js`; `SafeCInputHarness.jsx`.

Worked example and memory estimate

The example declares `age|integer|0..120|yes`, `temperature_c|decimal|-40..125|yes`, and `operator_note|text|80|no`. It produces an 83-line C scaffold. Text becomes `char operator_note[81]`; all console reads start with `fgets`, integers use checked `strtol`, decimals use checked `strtod` plus `isfinite`, and optional empty text is allowed. The conservative reported buffer budget is 128 transient input bytes plus 97 persistent bytes-8 each for the assumed `long` and `double`, and 81 for text-for 225 total. It explicitly excludes runtime stack/call overhead and therefore is not a measured compiler footprint. The example generates 16 boundary, invalid, optional, and overlength vectors.

Contract grammar and generation rules

Enter at most 20 rows as `C_identifier|integer/decimal/text|bounds|max|required yes/no`. Identifiers must be safe C identifiers, unique, and not reserved keywords. Integer/decimal bounds use `min..max`, must be finite and ordered; decimal fractions are supported. Text uses a maximum length from 1 through 512. Numeric vector epsilon is 1 for integer and $\max(0.001, \text{abs}(\text{max}-\text{min})/1000)$ for decimal. The transient buffer is $\max(128, \text{longestText}+2)$; persistent estimate adds text capacity plus terminator or an assumed eight bytes for each numeric variable.

Generate, copy into a disposable C project, compile with strict warnings, and run every emitted negative/boundary vector plus platform-specific fuzz cases. Review business bounds with the field owner and keep validated data away from format strings, commands, and memory indexes. The browser does not compile or execute C.

Build map tests and troubleshooting

`safeCInputHarness.js` owns identifier/bounds parsing, C escaping, vector generation, scaffold generation, and budget. The React screen displays errors and artifacts. Six focused tests cover example generation, vectors, budget, unsafe identifiers/bounds, fractional decimals, and string metacharacter escaping. If compilation differs, inspect compiler integer widths and includes; the eight-byte estimate is declared, not portable truth. Interview explanation: “I generated bounded input code together with adversarial vectors and an honest memory estimate, keeping compiler proof outside the browser.”

Final-UI screenshot: **capture pending after redesign acceptance.**

Lab 19 CSV Claim Stress Tester

- **Certificate connection:** Data Analysis with Python.
- **Real user/problem:** a small-business analyst or student has a claimed insight from a CSV and needs to know whether it survives basic data-quality and sensitivity checks.
- **Why built:** profiling/chart tools already exist; this tool tests one explicit business claim and creates a reproducible verification recipe.
- **Inputs:** bounded pasted CSV, numeric outcome column, group column, optional time column, and plain-language claim.
- **Decision logic:** robustly parses rows, applies missingness and sample-size gates, compares group means/medians/effect, repeats after IQR-based outlier trimming, and checks time-slice/subgroup consistency.

- **Outputs/artifacts:** Support, Fragile, or Reject verdict; quality/small-sample/missingness warnings; baseline and sensitivity evidence; and copyable pandas verification recipe.
- **Safety/privacy:** local only; no upload or API. The generated recipe does not embed the supplied rows.
- **Tests:** `tests/lab19-csv-claim-stress-tester.test.mjs` covers parsing, quality failures, supported/fragile/rejected claims, outlier sensitivity, time consistency, recipe generation, and bounds.
- **Limitations:** descriptive deterministic checks do not establish causality, statistical significance, representativeness, or a business decision.
- **Route/files:** `#/lab/csv-claim-stress-tester`; `csv-claim-stress-tester.js`; `CsvClaimStressTester.jsx`.

Worked example and exact evidence

The bundled 24-row CSV compares Premium and Standard `order_value` across Q1-Q4 and contains a Q4 Standard outlier of 160. All 24 rows are usable. Premium has $n=12$, mean 131.25, median 130.5; Standard has $n=12$, mean about 98.1667, median 93.5. The mean difference is about 33.0833 with standardized difference about 2.0697, supporting the “Premium higher” direction. IQR trimming removes one Standard row; the trimmed difference becomes about 38.7045 and still supports the claim. All four evaluable quarters support it, so the verdict is **Support** with one outlier warning. The generated pandas recipe names selected columns but never embeds pasted rows.

Parsing and verdict mechanics

Input is capped at 100,000 characters, 200 data rows, 2-20 columns, 2,000 characters per cell, and 12 groups. The local parser handles quoted commas, escaped quotes, embedded newlines, and a leading BOM; it rejects malformed shapes and duplicate headers. Claim interpretation must find exactly the two declared group names and one higher/lower direction. Baseline computes count, mean, median, and standardized mean gap. Sensitivity removes values outside $Q1-1.5 \times IQR$ and $Q3+1.5 \times IQR$ per group, then recomputes. Optional time slices repeat the comparison. Support requires baseline direction plus quality/sample, trimmed-direction, and available slice consistency gates; contradiction rejects, while dependence on an outlier, missingness, or inconsistent slices can make the result Fragile.

Paste non-sensitive CSV, select exact column headers, and state one directional comparison. Read quality before verdict, inspect group distributions/medians, compare trimmed evidence, and review every time slice. Export the pandas recipe and rerun against the governed source file before communicating a decision.

Build map tests and troubleshooting

`csv-claim-stress-tester.js` owns bounded CSV parsing, exact group/direction interpretation, descriptive statistics, IQR trimming, slices, verdict, and recipe. The React screen owns local paste and evidence views. Nine focused tests cover CSV quoting/shape, claim binding, trimming, support, rejection, fragile paths, and non-embedded recipe. If a group is not recognized, match its CSV spelling exactly; if Support becomes Fragile, follow warnings rather than editing the claim. Interview explanation: “I built a deterministic claim stress test that shows when an insight survives quality, outlier, and time checks-without implying causality or significance.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 20 LLM Data Contract Firewall

- **Certificate connection:** Generative AI / LLM Architecture and Data Preparation.
- **Real user/problem:** a data engineer needs to decide which declared fields can enter an LLM dataset before any records are processed.
- **Why built:** to create a fail-closed field-level contract for purpose, sensitivity, retention, and leakage tests rather than a generic data-preparation checklist.
- **Inputs:** metadata-only field rows describing name, intended task/purpose, classification, allowed use, and retention.
- **Decision logic:** maps every field to allow, redact, quarantine, or exclude, builds least-data processing stages, adds leakage gates, and withholds permissive output when the contract is incomplete.
- **Outputs/artifacts:** field decision ledger, least-data pipeline, leakage-test contract, strict JSON schema/JSONL record contract, and dataset-card text.
- **Safety/privacy:** accepts field metadata, not records; no LLM, file, API, secrets, production data, or compliance claim.
- **Tests:** `tests/lab20-llm-data-contract-firewall.test.mjs` covers classification/purpose decisions, fail-closed behavior, redaction/quarantine/exclusion, schema and dataset-card generation, leakage checks, and validation.
- **Limitations:** a metadata contract cannot verify actual records, consent, de-identification, retention enforcement, model memorization, legal basis, or dataset quality.
- **Route/files:** `#!/lab/llm-data-contract-firewall; llmDataContractFirewall.js; LlmDataContractFirewall.jsx`.

Worked example and exact field ledger

The example task is retrieval of approved guidance for an agent response, with seven metadata rows. `ticket_id`, `issue_summary`, `resolution_text`, and `public_product_name` are allowed; `customer_email` is redacted; `api_token` is quarantined; and `internal_agent_note` is excluded because retrieval is not in its allowed uses. Counts are allow 4, redact 1, quarantine 1, exclude 1. The strict JSON schema includes only allowed/redacted fields and sets `additionalProperties: false`; the dataset card explicitly says no records were inspected. Six processing stages and six leakage tests define fail-closed gates around the contract.

Decision precedence and schema rules

Enter at most 20 rows as `field_name|public/internal/personal/secret|comma-separated allowed uses|retention days`. Names start with a letter, contain only letters/numbers/underscores, and are unique; purposes are training, retrieval, or evaluation; retention is an integer from 0 through 3650. Rules run in order: if selected purpose is absent, exclude; otherwise secret is quarantine; personal over 90 days is exclude and otherwise redact; internal over 365 days is exclude; remaining fields allow. That precedence matters: a secret field disallowed for the purpose is excluded, not quarantined. Any validation failure withholds decisions/schema rather than defaulting to allow.

Use metadata only. Define the task and purpose, enter every candidate field, generate, and resolve quarantine/exclusion with data/security/legal owners. Implement the emitted schema and leakage gates

in the real pipeline, then test actual samples under governance; never paste records or secrets into this screen.

Build map tests and troubleshooting

`llmDataContractFirewall.js` owns grammar, decision precedence, stages, leakage tests, schema, and dataset card. The React screen renders and copies artifacts. Three focused tests prove all four actions, purpose/retention exclusion, and row/schema-metadata bounds. If a field action surprises you, inspect purpose membership before classification because purpose mismatch wins. Interview explanation: “I separated metadata policy from records, applied fail-closed precedence, and emitted an enforceable least-data schema plus leakage tests-not a compliance claim.” Final-UI screenshot: **capture pending after redesign acceptance**.

11 Labs 21 24 business finance and defensive security

Lab 21 Shortage Response Trade off Lab

- **Certificate connection:** Microeconomics Principles.
- **Real user/problem:** a small cooperative or shop has scarce essential inventory and must compare a temporary candidate-price policy with a baseline-price purchase cap without receiving a false “optimal” or legal answer.
- **Why built:** to keep access, unmet demand, consumer spend, and segment distribution beside revenue and margin.
- **Inputs:** essential-item description; up to 12 aggregate rows `segment|buyers|max units per buyer|willingness to pay`; available units; baseline price; unit cost; one candidate price; and a whole-number cap.
- **Decision logic:** candidate-price allocation admits qualifying willingness-to-pay segments and serves higher values first; capped allocation keeps the baseline price, limits buyer demand, and distributes scarcity proportionally. Both use the same inventory and report comparable measures.
- **Outputs/artifacts:** allocated units, revenue, gross-margin proxy, served buyers, unmet baseline demand, average spend, access coverage, segment service-rate gap, assumptions, and copyable owner memo.
- **Safety/privacy:** aggregate rows only; no person-level data or live prices. Willingness to pay is not need/fairness. Output is not pricing/legal advice and requires local essential-goods, anti-gouging, consumer-protection, and rationing review.
- **Tests:** `tests/lab21-shortage-response-tradeoff.test.mjs` covers both allocation paths, inventory/revenue/spend/access evidence, exclusions, zero access, bounds, unit cost, and memo limitations.
- **Limitations:** one-period static proxy; omits supply response, repeat visits, household need, taxes, enforcement, strategic behavior, and many fairness factors.
- **Route/files:** `#/lab/shortage-response-tradeoff`; `shortageResponseTradeoff.js`; `ShortageResponseTradeoffLab.jsx`.

Worked example and allocation math

The example has 100 one-week household water packs, baseline price 80, candidate price 110, unit cost 60, and cap 1. Demand is Families with children: 40 buyers $\times$ 2 at willingness 130; Older residents: 25 $\times$ 2 at 115; Other households: 55 $\times$ 1 at 95. Candidate pricing serves 80 family and 20 older-resident units, excluding the third segment: revenue 11,000, margin proxy 5,000, 60 served buyers, average spend about 183.33, access coverage 50%, and segment access gap 100%. The baseline-price cap proportionally allocates 35/20/45 units: revenue 8,000, margin 2,000, 100 served buyers, average spend 80, coverage 83.3%, and gap 7.5%. Both show 85 units of unmet original demand.

Validation accepts at most 12 unique aggregate segment rows and bounded nonnegative quantities/prices; the cap is whole-number. Candidate allocation filters `willingnessToPay >= candidatePrice`, sorts higher willingness first, and fills inventory. Cap allocation sets each segment demand to `buyers  $\times$  min(maxUnitsPerBuyer, cap)` and distributes inventory proportionally with deterministic remainder handling. Revenue is `units  $\times$  policy price`; margin proxy is `units  $\times$  (price-unitCost)`. Served buyers are inferred from allocated units and per-buyer limit, so they remain a proxy.

Run the comparison, read access and consumer-spend evidence beside revenue, inspect all five assumptions, and record legal/ethics review in the owner memo. `shortageResponseTradeoff.js` owns parsing, allocation, metrics, and memo; the JSX owns the local table/copy flow. Three focused tests pass for exact example differences, visible price exclusion, and input bounds. Troubleshooting: equal inventory does not imply equal people served; inspect each allocation rule. Interview explanation: “I made scarcity policy trade-offs visible without optimizing price or equating willingness to pay with need.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 22 LP Exit Rehearsal

- **Certificate connection:** Decentralized Finance (DeFi): The Future of Finance.
- **Real user/problem:** an educational DAO or club-treasury reviewer needs to rehearse an adverse constant-product liquidity-pool exit before a governance decision, without connecting a wallet.
- **Why built:** to put the hold benchmark, break-even fees, price impact, stop rules, and decision evidence around the AMM math.
- **Inputs:** hypothetical initial reserves, proposed deposit, fee assumption, adverse external-price multiplier, and percentage of the owned LP position to withdraw.
- **Decision logic:** uses the proportional deposit and reports unused tokens; derives LP ownership; holds $x \times y = k$ while rebalancing to the scenario price; values the withdrawal in token Y; compares it with holding; and models a small sample swap.
- **Outputs/artifacts:** owned/withdrawn balances, hold comparison, impermanent-loss value/percentage, fee break-even, curve price impact, separate fee-inclusive execution gap, stop triggers, and copyable pre-commitment memo.
- **Safety/privacy:** no wallet, token lookup, live price, approval, transaction, or returns promise; educational scenario only, not investment advice.
- **Tests:** `tests/lab22-lp-exit-rehearsal.test.mjs` covers adverse-price math, ownership, withdrawal, unused deposit, IL, break-even fees, separate impact/gap measures, stops, memo, source boundaries, and invalid numbers.

- **Limitations:** simplified full-range proportional liquidity; excludes accumulated fees, gas, tax, concentrated ranges, unusual tokens, hooks, vulnerabilities, liquidity changes, and execution conditions.
- **Route/files:** `#/lab/lp-exit-rehearsal`; `lpExitRehearsal.js`; `LpExitRehearsal.jsx`.

Worked example and formulas

Starting reserves are 1,000 X and 100,000 Y; deposit 100 X and 10,000 Y; fee 0.3%; adverse multiplier 0.5; withdraw 100%. The proportional deposit is fully used and owns $100 / (1,000 + 100) = 9.090909\%$. After deposit, $k = 1,100 \times 110,000 = 121,000,000$; at target price 50 Y/X, reserves rebalance to about 1,555.634919 X and 77,781.745931 Y. Withdrawal is 141.421356 X + 7,071.067812 Y, worth 14,142.135624 Y, versus holding value 15,000 Y: modeled IL -857.864376 or -5.719096% . Break-even fees are 857.864376 Y, 6.066017% of LP value. The sample 1%-of-X swap separates 0.990099% curve price impact from 1.284197% fee-inclusive execution gap.

Validation bounds all numeric scenario inputs and rejects impossible reserves/deposits, fee, multiplier, or withdrawal percentages. The engine uses only the proportional portion of an unbalanced deposit, preserves $x \times y = k$, values both positions at target price, and derives stop triggers. The example pauses because modeled IL is at least 5% and the price moved 2x. Curve impact and fee-inclusive gap are intentionally different labels.

Use the adverse scenario before any governance discussion, inspect unused deposits and all stop triggers, and copy the pre-commitment memo with owner/evidence/revisit fields.

`lpExitRehearsal.js` owns arithmetic and memo; the JSX has no wallet path. Four focused tests pass for the exact vector, unbalanced remainder/stop, swap labels/memo, and validation. Interview explanation: "This is a transparent exit rehearsal against holding, not a yield forecast or transaction interface." Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 23 Account Recovery Drill Composer

- **Certificate connection:** Cybersecurity Essentials.
- **Real user/problem:** a family or small-team security coordinator needs to rehearse device, phone, or primary-email loss without exposing credentials or initiating recovery.
- **Why built:** to detect correlated recovery dependencies and produce a secret-free tabletop package before a logout.
- **Inputs:** up to 20 category-only rows `account label | importance | primary MFA | recovery methods | notification channel`, plus device-loss, phone-loss, or email-loss scenarios.
- **Decision logic:** controlled categories model which recovery/notification methods fail in each scenario, detect single points/shared dependencies, and order verification and owned preparation actions.
- **Outputs/artifacts:** findings, no-secret tabletop steps, verification/notification checklist, generic emergency cards, and owner/evidence actions.
- **Safety/privacy:** rejects email-like labels, URLs, =, long digit sequences, usernames, passwords, recovery codes, secret answers, keys, and real addresses. It never starts recovery; users stop before submitting any provider action.
- **Tests:** `tests/lab23-account-recovery-drill.test.mjs` covers findings, single points, notification gaps, emergency cards, no-secret output, categories, missing scenarios, official sources, and unsafe label rejection including `username=deepan`.

- **Limitations:** cannot verify provider support, backup usability, notification delivery, process changes, or compromise. Owners must verify official provider paths safely.
- **Route/files:** `#/lab/account-recovery-drill`; `accountRecoveryDrill.js`; `AccountRecoveryDrillComposer.jsx`.

Worked example and dependency findings

The category-only example declares Family email (critical, authenticator app, saved code + backup device, push), Shared file storage (high, security key, backup device + provider support, email), and Event ticketing (medium, SMS, recovery phone, SMS), then selects device-, phone-, and email-loss. It creates nine emergency cards and six findings. Event ticketing under phone loss is a critical correlated single point because MFA and its only recovery method disappear; five high notification gaps are identified across failed channels, and the shared backup-device category creates one medium cross-account dependency. Seven tabletop steps and six checklist items repeatedly prohibit disclosing or entering secrets.

The parser allows at most 20 rows and controlled importance/MFA/recovery/notification categories. Generic labels reject email-like strings, URLs, long digit sequences, and characters such as `=`; at least one loss scenario is required. For each account/scenario, the model maps unavailable categories, surviving recovery methods, and independent notification status. It then adds shared-dependency findings across important accounts and ordered owner actions. This is declared-category reasoning, not provider validation.

Use fictional labels, run one scenario at a time, state the surviving category without its address/code/location, find the official provider route on a safe device, and stop before submission. `accountRecoveryDrill.js` owns parsing/mapping/cards/actions; JSX owns selection/copy. Four focused tests pass for findings, safe deterministic artifacts, unsafe/malformed rejection, and scenario/source requirements. Interview explanation: “I rehearsed correlated recovery failure while deliberately making the artifact useless for credential theft.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 24 Feature Misuse Contract

- **Certificate connection:** OWASP Top 10 - 2021.
- **Real user/problem:** a startup founder or developer is about to ship one web feature and needs feature-specific negative acceptance tests tied to its customer promise before merge.
- **Why built:** to convert roles and trust-boundary actions into tracked defensive tests, not an OWASP quiz, scanner, attack generator, or generic checklist.
- **Inputs:** feature/customer promise; 2-8 actor roles; selected data categories; and up to eight bounded actor/client/service/storage boundary actions.
- **Decision logic:** deterministically selects and prioritizes five to seven misuse stories. Each connects a broken promise to a safe scenario, relevant OWASP 2021 category, negative acceptance test, observable pass condition, control owner, and evidence.
- **Outputs/artifacts:** prioritized defensive misuse stories and copyable PR acceptance contract/checklist with boundary scope, assumptions, and merge evidence.
- **Safety/privacy:** local metadata only; no target URLs, credentials, secrets, payloads, exploit steps, CVSS, scanning, probing, or contacting real third parties.

- **Tests:** `tests/lab24-feature-misuse-contract.test.mjs` records 8 focused passes covering bounds, deterministic prioritization, complete contracts, sensitive data, safe remote-resource handling, PR output, limits, and official OWASP domains. Full Batch 6 suite: 126 passed.
- **Limitations:** planning artifacts cannot prove coverage, absence of vulnerabilities, control effectiveness, architecture correctness, or compliance; use synthetic test data and qualified review.
- **Route/files:** `#/lab/feature-misuse-contract`; `feature-misuse-contract.js`; `FeatureMisuseContract.jsx`.

Worked example and story contract

The example promises workspace owners can invite teammates to billing without exposing invoices/account details. Roles are owner, billing admin, member, and support agent; data categories are identity, contact, and payment; three boundaries cover browser→Invite API, Invite API→identity, and Billing API→database. The deterministic output contains six prioritized stories: two P1, three P2, and one P3, with owners spanning backend, security/data, identity, product, and operations/observability. Every story carries the broken promise, a safe non-exploit scenario, OWASP Top 10:2021 mapping, negative acceptance test, observable pass condition, owner, and evidence.

Promise and feature descriptions are bounded; roles are trimmed/deduplicated and limited to 2-8; selected data categories must be supported; boundary actions are limited to eight. Candidate stories derive from roles, sensitive categories, and described trust crossings, then are priority ordered and capped at five to seven. Sensitive data adds a cryptographic-failures test; remote-resource wording adds a controlled SSRF-policy test that explicitly avoids a real target. OWASP mapping is context, never a vulnerability finding.

Define the promise first, enumerate actual roles and trust crossings, generate, adapt each negative test to synthetic fixtures, and attach observable evidence before merge. `feature-misuse-contract.js` owns rules/prioritization/PR text; JSX renders it. Eight focused tests pass for bounds, deterministic completeness, sensitive/remote cases, safe copy contract, and official OWASP sources. Interview explanation: “I turned one product promise into owned negative acceptance evidence without scanning or generating exploits.” Final-UI screenshot: **capture pending after redesign acceptance**.

12 Labs 25 and 27 30 advanced defensive workflows

These five labs are implemented, documented, and manifest-marked functional. The earlier source-integration checkpoint recorded 151 cumulative automated tests. Their consolidated build/browser/mobile/console report was not closed in the batch-report directory, so these notes distinguish focused historical evidence from fresh integrated acceptance. This documentation update does not supply a new pass.

Lab 25 Share Link Afterlife Rehearsal

- **Certificate connection:** IT Security: Defense against the digital dark arts.
- **Real user/problem:** a small-team owner is about to share a sensitive document and needs to know what expiry/revocation can and cannot recover.
- **Why built:** to model the document's afterlife-downloaded copies, screenshots, caches, copied text, and forwarded metadata-rather than only provider permissions.

- **Inputs:** generic artifact label/classification; up to 12 role|business need|view/edit|required days rows; provider mode; permission; expiry; and yes/no residue capabilities.
- **Decision logic:** derives minimum specific-recipient access, required permission, and shortest supported expiry; calculates corrections and a proceed-after-corrections, safer-channel, or do-not-share gate; treats screenshots as non-revocable risk regardless of expectation.
- **Outputs/artifacts:** revocable/non-revocable residue ledger, owner/evidence rows, expiry and former-recipient verification checks, and copyable pre-share/expiry contract.
- **Safety/privacy:** rejects URLs, email-like identities, token-like links, and identifying rows; accepts no file/content/link/token and performs no provider action. Acknowledgement is not proof of deletion.
- **Tests:** tests/lab25-share-link-afterlife.test.mjs records 4/4 focused passes covering access/expiry narrowing, residue, restricted-material blocking, permission downgrade, and invalid metadata. JSX transformation passed.
- **Limitations:** cannot inspect effective permissions, inheritance, guests, sync, logs, retention, legal holds, caches, screenshots, or recipient behavior; not confidentiality/compliance proof.
- **Route/files:** #/lab/share-link-afterlife; shareLinkAfterlife.js; ShareLinkAfterlifeRehearsal.jsx.

Worked example and afterlife gate

The built-in share scenario recommends named specific recipients, edit permission, and 10-day expiry. Because the entered proposal uses organization-wide access and 30 days, its gate is **proceed-after-corrections** with two explicit corrections: narrow access to named recipients and reduce expiry from 30 to 10 days. Six lifecycle ledger entries and six verification checks cover pre-share, active access, expiry/revocation, download/copy residue, cached/offline copies, and accountable follow-up. The tool distinguishes provider-revocable access from copies that revocation cannot recover.

Recipient roles are bounded to 12 generic rows and must not contain identities or contact details. The engine combines material sensitivity, recipient need, requested permission/scope/expiry, download/copy behavior, and explicit residue acknowledgement. Unnecessary edit permission triggers downgrade; restricted material with unavoidable residue can produce do-not-share even when expiry exists. Expiry changes provider access state-it does not erase an already downloaded, copied, cached, or photographed artifact.

Run before sharing, apply every correction in the real provider, confirm named recipients and least permission, agree residue handling/owner, and perform verification after expiry or revocation. `shareLinkAfterlife.js` owns validation, gate, lifecycle ledger, verification, and contract. Four focused tests pass for example narrowing/residue, restricted do-not-share, permission downgrade, and unsafe/malformed input. Interview explanation: “I modeled what link controls can revoke and what information residue survives, preventing expiry from being presented as deletion.” Final-UI screenshot: **capture pending after redesign acceptance.**

Lab 27 Proof to Interview Compiler

- **Certificate connection:** Put It to Work: Prepare for Cybersecurity Jobs.
- **Real user/problem:** a cybersecurity student preparing for an internship has projects and tests but needs every resume/interview statement traceable to evidence.
- **Why built:** to withhold unsupported achievements instead of polishing them into plausible-sounding claims.

- **Inputs:** generic target role; up to eight target tasks; project situation/action/outcome; up to 16 artifact-id|type|what it proves rows; and up to 12 candidate skills.
- **Decision logic:** deterministic term matching links claims/tasks to artifact descriptions. Claims become Backed, Needs evidence, or Do not claim; unsupported numbers and absolute proficiency language are withheld. STAR action/result citations are calculated independently.
- **Outputs/artifacts:** claim ledger, NICE-aligned task map, citation-bearing resume bullets only for backed claims, STAR answer, interviewer evidence packet, and gap drill.
- **Safety/privacy:** rejects identities, emails, URLs, employer/repository links, and unsupported artifact types. It never invents metrics, employers, responsibilities, outcomes, certifications, or hiring promises.
- **Tests:** tests/lab27-proof-to-interview-compiler.test.mjs records 5/5 focused passes covering supported citations, unsupported numeric claims, task/STAR/packet/gap output, no-bullet rules, and unsafe metadata. JSX transformation passed.
- **Limitations:** term overlap is not semantic verification; artifact descriptions are user assertions and the tool cannot inspect, run, authenticate, or grade them or predict hiring.
- **Route/files:** #/lab/proof-to-interview-compiler; proofToInterviewCompiler.js; ProofToInterviewCompiler.jsx.

Worked example and evidence gate

The example targets a cybersecurity analyst internship and provides two tasks, a situation/action/outcome, and three artifacts: LOG-SCAN-CODE, LOG-SCAN-TEST, and LOG-SCAN-REPORT. Claims that a Python parser was built and automated tests were written are backed and become two citation-bearing bullets. “Reduced security incidents by 80%” becomes **do-not-claim** because neither artifact descriptions nor the supplied outcome prove that number; it produces a gap drill requesting a code/test/report/demo artifact with expected, observed, and limited results. The task map, STAR answer, and evidence packet preserve the same IDs.

Input caps are eight task statements, 16 artifacts, and 12 claims. Artifact rows require unique IDs, a supported evidence type, and a bounded description; links and unsafe types are rejected. Claims are normalized and matched to artifact language. Numeric claims require numeric support in artifact description plus outcome—an outcome alone cannot substitute for artifact proof. Only backed claims release resume bullets; unsupported claims remain in the gap drill.

Enter what was actually built, cite reviewable artifact IDs, generate, and manually verify each bullet against the repository/test/report before resume use. proofToInterviewCompiler.js owns parsing, overlap/evidence rules, tasks, bullets, STAR, packet, and gaps. Five focused tests pass for citations/metric withholding, task/STAR preservation, unsupported claims, outcome insufficiency, and validation. Interview explanation: “I made career claims traceable to artifacts and fail closed on unsupported metrics.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 28 Containment Side Effect Ledger

- **Certificate connection:** Introduction to Cybersecurity Tools & Cyberattacks.
- **Real user/problem:** a junior SOC analyst has a plausible alert and candidate containment actions but lacks authority to trade customer availability for security without evidence and owner review.
- **Why built:** to put evidence confidence, customer-promise harm, reversibility, rollback proof, and authorization in one pre-action ledger.

- **Inputs:** generic alert claim; affected service/customer promise; up to 20 fact | source category | confidence rows; and up to 12 action | reversibility | service harm | owner rows.
- **Decision logic:** requires at least one high-confidence fact and two source categories before high-harm action becomes reviewable; otherwise blocks/escalates. Irreversible actions are always “escalate only.”
- **Outputs/artifacts:** per-action disposition, customer side effect, evidence to collect, authorization gate, recovery proof, escalation trigger, owner, evidence gaps, and copyable analyst handoff.
- **Safety/privacy:** rejects URLs, address/hash-like indicators, credentials, payload/script/query/command content, and contact-address owners. It connects to no telemetry/system and executes or recommends no real containment action.
- **Tests:** tests/lab28-containment-side-effect-ledger.test.mjs records 4/4 focused passes for complete ledgers, never recommending irreversible action, blocking weak-evidence high harm, and unsafe-input rejection. JSX parse and diff check passed.
- **Limitations:** does not establish incident truth, evidence authenticity, action effectiveness, acceptable customer harm, or authorization; confidence/diversity thresholds are educational baselines.
- **Route/files:** #/lab/containment-side-effect-ledger; containmentSideEffectLedger.js; ContainmentSideEffectLedger.jsx.

Worked example and disposition rules

The example alert is unusual sign-in failures affecting the promise that customers can reach support. Evidence is high-confidence application failure-rate change plus medium identity-provider and customer-report signals. Candidate actions are reversible step-up verification/medium harm, partial disable-all-sign-ins/high harm, and irreversible delete-accounts/high harm. No deterministic evidence gap fires. The three dispositions are respectively **reviewable**, **escalate-for-review**, and **escalate-only**; the irreversible candidate can never be recommended. Every entry includes customer side effect, pre-action evidence, authorization gate, rollback/recovery proof, escalation trigger, and generic owner.

Metadata parsers reject URLs, address/hash-like indicators, credentials/secrets, payload/query/command content, and contact-address owners. A high-harm action requires at least one high-confidence fact and two source categories before review; insufficient evidence blocks it. Irreversible always escalates only. These thresholds organize review and do not establish that an incident exists.

Use generic evidence claims, review evidence gaps first, and take the ledger only to the authorized incident commander/service/legal/recovery owners. `containmentSideEffectLedger.js` owns safety parsing, evidence gates, disposition and handoff; `JSX` owns local rendering/copy. Four focused tests pass for complete ledger, irreversible invariant, unsupported high-harm block, and unsafe input rejection. Interview explanation: “I bound containment proposals to evidence, customer harm, authorization, and recovery proof rather than optimizing for immediate shutdown.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 29 Detection Contract Drift Guard

- **Certificate connection:** Automate Cybersecurity Tasks with Python.
- **Real user/problem:** a SOC engineer needs to detect when a producer schema change silently changes the meaning of a Python-based detection.
- **Why built:** to compare one detection claim's field/type/unit/time assumptions and create a regression contract without ingesting logs or connecting to a SIEM.

- **Inputs:** detection claim; up to 50 `field|required yes/no|type|unit` assumptions; old/new schema rows `field|type|unit`; and a trailing `?` convention for newly optional new-schema fields.
- **Decision logic:** classifies required disappearance/type incompatibility as invalid drift; optional disappearance/unit change as silent drift; newly optional/additional/compatible-but-changed fields as widened drift. Each impact receives repair evidence and bounded vectors.
- **Outputs/artifacts:** drift ledger, affected claim, repair contract, valid/missing/wrong-type vectors, and copyable standard-library `unittest` Python harness with a 64-field guard.
- **Safety/privacy:** metadata only; rejects queries, executable content, URLs, credentials, secrets, records, and unsupported identifiers. It does not run Python or connect to a source.
- **Tests:** `tests/lab29-detection-contract-drift-guard.test.mjs` records 4/4 focused passes for classifications, repair/vectors, bounded standard-library source, and validation. Generated Python passed in-memory `compile()` syntax only; it was not executed.
- **Limitations:** cannot prove event completeness, values, time correctness, semantics, false-positive/negative behavior, or detection coverage; `?` is a lab convention.
- **Route/files:** `#!/lab/detection-contract-drift-guard; detectionContractDriftGuard.js; DetectionContractDriftGuard.jsx`.

Worked example and exact drift package

The example claim flags repeated failed sign-ins per account in five minutes. Its old/new metadata produces six impacts: `account_id` newly optional (widened), `outcome` removed (invalid), `event_time` unit changes from UTC ISO-8601 to milliseconds (silent), `failure_count` changes number→string (invalid), and new `result` and `region` fields widen the surface. Six repair entries pair action with proof. Nine regression vectors include one valid contract, four missing-required cases, and four wrong-type cases. The generated standard-library Python `unittest` harness caps events at 64 fields and uses `datetime.fromisoformat`; it has no log reader or SIEM connection.

The lab accepts one bounded safe claim and up to 50 rows in each assumptions/old/new schema table. Assumptions use `field|required yes/no|type|unit`; schema uses `field|type|unit`, with a trailing `?` lab convention for newly optional fields. Types and units are controlled. Missing required/type mismatch are invalid; optional removal/unit change may be silent; new optionality or undeclared fields are widened. Classification is metadata comparison, not detector execution.

Compare producer contracts, resolve every impact, copy/review the harness, then run it only in authorized development and add threshold/window/domain tests. `detectionContractDriftGuard.js` owns parsers, classification, repair, vectors, and Python source. Four focused tests pass for classifications, deterministic vectors, `stdlib/64` guard, and grammar safety. Interview explanation: “I caught semantic schema drift that can leave Python running while changing detection meaning.” Final-UI screenshot: **capture pending after redesign acceptance**.

Lab 30 Assurance Change Shockwave Mapper

- **Certificate connection:** Play It Safe: Manage Security Risks.
- **Real user/problem:** a small-company security lead has a vendor, system, configuration, or owner change and needs to know which evidence-backed control claims and accepted-risk decisions require review.
- **Why built:** to start from one operational change and trace its evidence shockwave, not create another static risk register or audit dashboard.

- **Inputs:** change category/description; up to 20 claim-id|promise|owner|review cadence days rows; up to 40 evidence-id|claim-id|type|age days|dependency categories rows; and up to 20 risk-id|claim-id|days until expiry rows.
- **Decision logic:** matching change dependency invalidates that evidence chain; age at/over cadence needs review; otherwise evidence is current from supplied metadata. Claims inherit the worst linked evidence state. Risk decisions are expired at ≤ 0 days, due soon at 1-30, active above 30. Renewal order uses visible state/expiry/overdue/ID tie rules and no opaque score.
- **Outputs/artifacts:** claim dispositions, evidence shockwave, renewal instructions/order, risk-decision windows, owner/evidence needs, assumptions, and copyable executive memo.
- **Safety/privacy:** generic metadata only; rejects unknown links, URLs, and credential-like values; no system/evidence inspection or integration. Never certifies compliance, approves an audit/change, accepts risk, or renews a decision.
- **Tests:** tests/lab30-assurance-change-shockwave.test.mjs records 8/8 focused passes for parsing, safety validation, dependency propagation, cadence boundary, expiry labels, deterministic no-score order, safe memo language, and NIST source domains. JSX parse and diff check passed.
- **Limitations:** only as complete as supplied metadata; cannot prove control failure/effectiveness, evidence authenticity/sufficiency, complete dependencies, or appropriate risk acceptance.
- **Route/files:** #/lab/assurance-change-shockwave; assuranceChangeShockwave.js; AssuranceChangeShockwaveMapper.jsx.

Worked example and renewal order

The example vendor change affects four payroll promises and five evidence records. EV-01 and EV-03 depend on vendor and are invalidated, so CLM-01 and CLM-02 are invalidated. EV-04 is 40 days old against a 30-day cadence, so CLM-03 needs review; CLM-04 remains current from supplied metadata. Summary is 2 invalidated, 1 needs review, 1 current. Risk windows are 14 days due-soon, -3 expired, and 120 active. Renewal order is CLM-02, CLM-01, CLM-03, CLM-04 because status is considered first, then expired/due-soon decision, overdue days, and claim ID.

Input limits are 20 claims, 40 evidence rows, and 20 accepted risks. Evidence matching the selected change category is invalidated; otherwise age $\geq$ claim cadence needs review. A claim inherits invalidation from any linked evidence, otherwise needs review for missing/stale evidence, else current. Risk expiry is expired at ≤ 0 , due soon at 1-30, active above 30. Category matching invalidates evidence sufficiency, not the underlying control, and no numeric risk score is produced.

Enter generic metadata, run after a vendor/system/config/owner change, inspect dependency propagation, and assign each renewal to the declared owner in the authorized evidence system. assuranceChangeShockwave.js owns parsers/links/states/order/memo; JSX owns views/copy. Eight focused tests pass for typed parsing, unsafe links, propagation, cadence boundary, expiry states, ordering, non-certification memo, and NIST sources. Interview explanation: "I trace a concrete change through evidence, promises, cadence, and expiring decisions without certifying compliance or accepting risk." Final-UI screenshot: **capture pending after redesign acceptance**.

13 Verification and acceptance

Automated gate

For every code change:

- 1 run the focused test file;
- 2 validate malformed, boundary, and deterministic paths, not only the sample;
- 3 run `pnpm run test` to validate the manifest and full Node suite;
- 4 run the relevant production build;
- 5 for the current portfolio-only pass, run `pnpm run build:portfolio`; use `pnpm run build` only for separately authorized all-surface maintenance because it rewrites the excluded Labs and Timber build outputs.

The earlier Batch 7 source-integration checkpoint recorded **151 passing tests**. The coordinator also reported **151 passing tests** and a successful `pnpm run build:site` for the independent local fixes on 2026-08-31. Neither count is a permanent promise or evidence that the pending redesign is accepted. The consolidated historical batch reports available for this documentation audit run through Batch 6; preserve that provenance gap and record route-level browser results for new UI changes separately.

Browser gate

For a realistic example on every changed route:

- load the direct hash route, not only catalog navigation;
- use example/reset and the primary calculate/generate action;
- trigger one meaningful validation or safety path;
- verify copy/download/print behavior when the lab promises it;
- reload when persistence is part of the product;
- inspect the console for warnings/errors;
- check keyboard focus, labels, instructions, and result announcements;
- inspect at a 390 × 844 viewport and a desktop viewport;
- confirm dense tables are locally scrollable and do not create page-level overflow.

The completed batch reports record historical browser and console checks through Labs 01-24 and 26. Do not extend those results to undocumented routes or new UI changes. A current browser pass must identify the revision, route, input/workflow, viewport, and observed outcome.

Functional acceptance questions

A lab does not pass merely because it renders. Ask:

- Can the intended user name the decision or artifact they get?
- Does the example produce something they can reuse outside the page?
- Are calculations and tie-breaks inspectable?
- Does invalid input withhold misleading output?
- Are privacy and authority boundaries visible before action?

- Does the output avoid claims the model cannot establish?
- Can a reviewer reproduce important results from tests?

CI

`.github/workflows/frontend-tests.yml` uses Node 22, pnpm 11.19.0, a frozen lockfile install, and `pnpm run build` on pushes and pull requests to `main/master`. This is the repository's clearest currently recorded runtime reference even though `package.json` itself has no `engines` field.

The separate `.github/workflows/python-tests.yml` runs Python 3.12 and legacy Python unit discovery. That workflow belongs to older repository utilities and should not be confused with the Node Applied Labs suite.

14 Deployment and rollback

Public surfaces

The existing unified release uses the following routes. These are the current source/configuration mappings and supplied release URLs; the documentation correction itself is not a fresh production verification:

Surface	Build/source boundary	Release route
Portfolio	<code>portfolio/</code> from <code>portfolio-src/</code>	<code>https://deepan-karthick-portfolio.industrious-keyboard.workers.dev/</code>
Applied Labs	<code>labs/</code> from <code>labs-src/</code>	<code>https://deepan-karthick-portfolio.industrious-keyboard.workers.dev/labs/</code>
Timber demo	<code>timber-demo/</code> from <code>timber-src/</code>	<code>https://deepan-karthick-portfolio.industrious-keyboard.workers.dev/timber-demo/</code>
Printing ERP demo	separate private source/build	<code>https://lakshmipriya-erp-demo.industrious-keyboard.workers.dev/owner</code>

`wrangler.jsonc` configures the `deepan-karthick-portfolio` Worker to serve `./site-dist`. `scripts/assemble_cloudflare_site.mjs` puts the portfolio at the package root, keeps `/portfolio/` as a compatibility copy, adds the labs and Timber directories, and copies selected PDFs. The three public surfaces are one static package, not three separately deployed projects. ERP remains separate; packaging this repository neither updates its deployment nor proves its private backend health.

The intended public source repository is

`https://github.com/darkraigamer212-cmd/deepan-karthick-portfolio-labs`, verified in the local `portfolio-origin` remote configuration. The local `origin` still names `career-application-kit`. Check the target remote before publishing; this documentation pass does not claim that any new changes were pushed.

Deployment sequence

This is the general full-site release procedure, **not authorization to deploy the current checkpoint**. The active pass is portfolio-only, with production release pending. Any later shared-package release must deliberately preserve accepted Labs and Timber assets; do not publish unrelated local changes through an all-surface rebuild. ERP has its own independent release boundary and is excluded.

- 1 Ensure the working tree contains no client database, real export, credentials, or unreviewed generated artifact.
- 2 Run `pnpm install --frozen-lockfile` in CI or the documented equivalent.
- 3 Run `pnpm run build:site` to test, build all three surfaces, and assemble `site-dist/`.
- 4 Inspect the assembled entry files, assets, route structure, and selected PDFs. Regenerate and visually check the manual separately when its source changes; the Node build will otherwise package the old PDF.
- 5 Publish the assembled package using the authenticated Cloudflare deployment workflow and this repository's `wrangler.jsonc`. ERP uses its own separate configuration and release workflow.
- 6 Verify the portfolio home, a direct Applied Labs hash route, and the Timber sample/save/reopen/print path. Check the ERP link and its public workflow separately when it is in the accepted release scope.
- 7 Record the commit, deployment/version ID, URL, time, and accepted checks in project control.
- 8 Keep the previous known-good deployment available until smoke tests pass.

`pnpm run site:assemble` is a packaging-only shortcut for already accepted build outputs. Neither it nor `pnpm run build:site` publishes to Cloudflare. A deployment command returning successfully is not sufficient acceptance: record the actual revision/version and observed workflows. Authentication, account selection, and any required permission remain separate from local build instructions.

Rollback

- Re-promote the last known-good static deployment/version rather than editing built files in production.
- For a bad manifest or route integration, revert the specific source change, rebuild all surfaces, and re-run the direct-route smoke tests.
- For Timber, never “repair” a formula in production without restoring the golden business vectors.
- For ERP, public demo rollback must not enable live mode or reuse live secrets. Version `4bd9ebd0-69d8-4c3f-b238-947fde815cc5` is a historical Batch 1B record, not an instruction to roll back to it without checking the current deployment history.
- Document the failure symptom, affected route/version, rollback result, and follow-up owner.

15 Maintenance guide

Adding or changing a lab

- 1 Start with the certificate concept, real user, problem, and reusable artifact.
- 2 Check adjacent/common tools and write the exact workflow differentiator; do not claim worldwide novelty.
- 3 Define bounded inputs, deterministic rules, safety limits, and validation before UI work.
- 4 Keep pure logic in a lab-specific `.js` module and UI in `.jsx`.
- 5 Add or update a unique focused test in `tests/`.

- 6 Add the lazy route in `labs-src/main.jsx` and the canonical entry in `project-manifest.json` only through the shared integration workflow.
- 7 Add/update the project-control lab document with source boundaries and limitations.
- 8 Run focused tests, full suite, build, realistic browser flow, mobile, keyboard, and console checks.
- 9 Update status/report only after evidence exists.

Dependency maintenance

- Keep `pnpm-lock.yaml` reviewed and committed with dependency changes.
- Run the full combined build after React, Vite, animation, or plugin changes.
- Treat security audit output as a triage input, not permission for an untested bulk upgrade.
- Recheck the separate ERP's historical advisory findings before a dedicated repair/test pass; this manual does not establish current advisory counts.
- Avoid adding APIs or large runtimes to labs whose value depends on reliable no-key local execution.

Data hygiene

- Never commit `.db`, `.db-wal`, `.db-shm`, `profile_data.json`, secrets, or imported client ZIP content.
- Keep examples fictional and visibly labelled.
- Do not paste real credentials, customer identities, logs, telemetry, wallet data, recovery codes, account addresses, or production evidence into labs.
- Inspect exported/copied artifacts so they contain only intended generic metadata.
- Browser-local storage is not a secure vault, backup, or cross-device database.

Documentation maintenance

Update this manual only from evidence. If a new deployment is made, add its URL, platform, commit, version ID, and smoke-test result. If the code changes a formula or decision threshold, update the lab section and tests together. If a detail cannot be verified, write **unknown** or **pending** rather than copying an assumption from a screenshot or old README.

Use `docs/project-control/redesign-guide-coverage.md` for the original chapter template, source-of-truth policy, and acceptance checks. After changing the accepted Markdown, regenerate PDF/DOCX with `scripts/build_complete_manual.py`, inspect rendered outputs, then deliberately package/publish the public guide. The September 4 generation updates the local artifacts, not the existing public deployment. Do not expose private imports or the entire docs tree to make a guide download work.

16 Troubleshooting

A lab route shows the catalog or not found state

- Confirm the route is exactly `#/lab/<manifest-slug>`.
- Confirm the slug matches `docs/project-control/project-manifest.json` and the registry in `labs-src/main.jsx`.
- Confirm the lazy import filename and exported React component match.

- Run the Applied Labs build to catch missing modules or JSX errors.
- For normal path-based SPA deep links on other applications, confirm the host's fallback; hash lab routes should not require a server rewrite.

Manifest validation fails

- Run `node scripts/validate_project_manifest.mjs` directly.
- Check duplicate IDs/slugs, total count, status values, certificate mapping, and flagship count.
- Do not “fix” the validator to accept an accidental manifest shape; reconcile source and intended schema.

A test passes alone but full build fails

- Run `pnpm run test`, then the specific `build:*` command to isolate whether the problem is manifest, cross-test state, JSX, import path, or bundling.
- Check filename case; Linux CI is case-sensitive even when the Windows workspace is forgiving.
- Check for shared-file conflicts from parallel work.
- Use the Vite error's first source location, not only its final stack frame.

The page overflows on mobile

- Inspect the page's `scrollWidth` versus `clientWidth` at 390 px.
- Check grid/flex children for missing `min-width: 0`.
- Contain wide tables within their own horizontal scroll region.
- Avoid fixed pixel widths in copied code/memo panels.
- Re-run the realistic workflow after the CSS fix; an empty form may not expose result-table overflow.

Copy or download does not work

- Clipboard APIs may require a secure context and browser permission; provide visible success/failure state and manual-select fallback where possible.
- For SVG export, verify XML escaping, metadata, filename, and Blob/object-URL cleanup.
- For print/PDF, use the browser print dialog; do not claim a PDF was created until the user saves it.

Browser local records disappear or corrupt

- Confirm the same browser/profile/origin is in use.
- Browser storage can be cleared by the user or policy and is not a backup.
- Use the product's reset/recovery behavior; never connect the public demo to a private database to “restore” sample state.
- For Timber, test sample → save → reset → reopen again after storage changes.

Timber totals differ from an ordinary calculator

- Confirm business inches use $\text{millimetres} \div 20$.
- Confirm positive dimensions round upward to 0.5 inch with a 1.5-inch minimum.
- Confirm CFT uses rounded billing dimensions while M3 uses raw millimetres and actual metric length.
- Confirm line amounts and invoice totals apply the specified rounding order.
- Run `node --test tests/timber-domain.test.mjs` before changing code.

ERP dashboard hangs or shows no demo data

- Confirm the public deployment is explicitly in demo mode.
- Confirm demo initialization creates no Supabase client/request.
- Inspect corrupt local demo storage recovery.
- Do not add a temporary owner bypass or synthetic fallback to live mode.
- Compare the deployment with the recorded accepted Cloudflare version and private ERP runbook.

Deployment appears successful but a direct route fails

- Test the exact production URL after deployment, including /owner for ERP and a direct lab hash route.
- For path-routed SPAs, configure a static-asset fallback to the app entry.
- Record the deployed commit/version; otherwise it is impossible to distinguish a stale deployment from a source bug.

17 How to explain the project in an interview

Thirty second version

I converted 30 course certificates into a single Applied Labs product and kept two larger business applications as flagships. I used a functional-first process: each lab has bounded input, transparent local logic, a useful artifact, validation, tests, and honest limitations before any visual redesign. The two flagships demonstrate privacy-aware demo architecture-browser-local synthetic modes for public review and strict separation from private business data.

Two minute architecture version

Explain four decisions:

- 1 **One catalog, 30 lazy routes:** easier deployment and maintenance than 30 sites, while every lab remains independently testable.
- 2 **Pure logic separated from React:** Node tests can verify formulas, parsing, tie-breaks, and safety gates without browser automation.
- 3 **Local-first demonstrations:** no API key, wallet, cloud account, scanner, or secret is needed for the core result; demos remain reliable and privacy-preserving.
- 4 **Truthful product boundaries:** simulated attention is not a language model, SVG generation is not a trained GAN, planning tools do not certify compliance, and checkpoint test counts are not user-impact metrics.

Flagship story

For Printing ERP, describe the original failure (public dashboard stuck on empty loading skeletons), the architectural fix (explicit demo/live runtime split), the security rule (demo never touches Supabase; live authentication/configuration fails closed), persistence verification, and Cloudflare direct-route deployment. Preserve §5's qualification about live service missing-table fallbacks; do not claim every backend failure is universally fail-closed.

For Timber, explain that the hardest part was preserving a client's nonstandard but intentional business-inch rule while adding a complete billing domain. Show the difference between rounded CFT billing dimensions and raw-millimetre M3, then explain synthetic local storage and why the private SQLite database was excluded.

Lab story pattern

Use this structure for any lab:

- 1 “The certificate concept was ...”
- 2 “The real user was ... and their decision/problem was ...”
- 3 “I avoided a generic quiz/calculator by making the output ...”
- 4 “The deterministic rule is ...”
- 5 “The main safety/privacy boundary is ...”
- 6 “The focused test proves ...”
- 7 “It still cannot prove ...”

Example for Lab 24:

OWASP lists security categories, but a startup developer still needs merge-ready tests for one feature. The lab starts from a customer promise and trust-boundary actions, generates safe misuse stories, maps them to relevant OWASP 2021 categories, and produces observable negative acceptance tests with owners and evidence. It never scans or generates exploit payloads, and the output does not prove the feature is secure.

Evidence to show

- a live flagship workflow and its privacy statement;
- one Applied Lab's example, validation path, and copied/exported artifact;
- the pure domain module next to its focused Node test;
- the canonical manifest and lazy route;
- a batch report showing what passed and what remained pending;
- a limitation statement that prevents overclaiming.

Never say “30 production AI systems,” “world-first,” “fully secure,” “compliant,” or “deployed everywhere.” Say “30 functional, deterministic applied prototypes,” then distinguish the existing implementation/release from pending redesign and fresh acceptance work.

18 Appendices

Appendix A canonical lab routes sources and tests

All component/model paths below are under `labs-src/labs/`; all tests are under `tests/`.

#	Route slug	Domain module	React component	Focused test
01	ai-workflow-canvas	aiWorkflowCanvas.js	AIWorkflowCanvas.jsx	lab01-ai-workflow-canvas.test.mjs

#	Route slug	Domain module	React component	Focused test
02	mini-rag-studio	miniRagStudio.js	MiniRagStudio.jsx	lab02-mini-rag-studio.test.mjs
03	gan-latent-gallery	ganLatentModel.js	GanLatentGallery.jsx	lab03-gan-latent.test.mjs
04	cnn-feature-explorer	cnnConvolution.js	CnnFeatureExplorer.jsx	lab04-cnn-convolution.test.mjs
05	attention-text-explorer	attention-text-explorer.js	AttentionTextExplorer.jsx	attention-text-explorer.test.mjs
06	neural-network-playground	neuralNetworkPlayground.js	NeuralNetworkPlayground.jsx	lab06-neural-network-playground.test.mjs
07	genai-lifecycle-explorer	genAiLifecycleExplorer.js	GenAiLifecycleExplorer.jsx	lab07-genai-lifecycle-explorer.test.mjs
08	ai-opportunity-scorer	aiOpportunityScorer.js	AiOpportunityScorer.jsx	lab08-ai-opportunity-scorer.test.mjs
09	prompt-workbench	promptWorkbench.js	PromptWorkbench.jsx	lab09-prompt-workbench.test.mjs
10	ml-model-lab	ml-model-lab.js	MlModelLab.jsx	lab10-ml-model-lab.test.mjs
11	cloud-regret-premortem	cloudRegretPremortem.js	CloudRegretPremortem.jsx	lab11-cloud-regret-premortem.test.mjs
12	aws-outage-storyboard	awsOutageStoryboard.js	AwsCustomerJourneyOutageStoryboard.jsx	lab12-aws-outage-storyboard.test.mjs
13	azure-access-handoff	azureAccessHandoff.js	AzureAccessHandoffSimulator.jsx	lab13-azure-access-handoff.test.mjs
14	gcp-promise-ledger	gcp-promise-ledger.js	GcpTransformationPromiseLedger.jsx	lab14-gcp-promise-ledger.test.mjs
15	responsive-constraint-handoff	responsiveConstraintHandoff.js	ResponsiveConstraintHandoff.jsx	lab15-responsive-constraint-handoff.test.mjs
16	exception-first-python-automator	exceptionFirstPythonAutomator.js	ExceptionFirstPythonAutomator.jsx	lab16-exception-first-python-automator.test.mjs
17	queue-fairness-replay	queueFairnessReplay.js	QueueFairnessReplay.jsx	lab17-queue-fairness-replay.test.mjs
18	safe-c-input-harness	safeCInputHarness.js	SafeCInputHarness.jsx	lab18-safe-c-input-harness.test.mjs
19	csv-claim-stress-tester	csv-claim-stress-tester.js	CsvClaimStressTester.jsx	lab19-csv-claim-stress-tester.test.mjs
20	llm-data-contract-firewall	llmDataContractFirewall.js	LlmDataContractFirewall.jsx	lab20-llm-data-contract-firewall.test.mjs
21	shortage-response-tradeoff	shortageResponseTradeoff.js	ShortageResponseTradeoffLab.jsx	lab21-shortage-response-tradeoff.test.mjs
22	lp-exit-rehearsal	lpExitRehearsal.js	LpExitRehearsal.jsx	lab22-lp-exit-rehearsal.test.mjs
23	account-recovery-drill	accountRecoveryDrill.js	AccountRecoveryDrillCompiler.jsx	lab23-account-recovery-drill.test.mjs
24	feature-misuse-contract	feature-misuse-contract.js	FeatureMisuseContract.jsx	lab24-feature-misuse-contract.test.mjs

#	Route slug	Domain module	React component	Focused test
25	share-link-after-life	shareLinkAfterlife.js	ShareLinkAfterlifeRehearsal.jsx	lab25-share-link-afterlife.test.mjs
26	network-change-rollback	networkChangeRollback.js	NetworkChangeRollbackComposer.jsx	lab26-network-change-rollback.test.mjs
27	proof-to-interview-compiler	proofToInterviewCompiler.js	ProofToInterviewCompiler.jsx	lab27-proof-to-interview-compiler.test.mjs
28	containment-side-effect-ledger	containmentSideEffectLedger.js	ContainmentSideEffectLedger.jsx	lab28-containment-side-effect-ledger.test.mjs
29	detection-contract-drift-guard	detectionContractDriftGuard.js	DetectionContractDriftGuard.jsx	lab29-detection-contract-drift-guard.test.mjs
30	assurance-change-shockwave	assuranceChangeShockwave.js	AssuranceChangeShockwaveMapper.jsx	lab30-assurance-change-shockwave.test.mjs

Appendix B command reference

```
# Install
pnpm install --frozen-lockfile

# Develop one surface
pnpm run dev:portfolio
pnpm run dev:labs
pnpm run dev:timber

# Validate canonical manifest only
node scripts/validate_project_manifest.mjs

# Full Node/manifest test gate
pnpm run test

# Example focused tests
node --test tests/lab01-ai-workflow-canvas.test.mjs
node --test tests/lab19-csv-claim-stress-tester.test.mjs
node --test tests/lab30-assurance-change-shockwave.test.mjs
node --test tests/timber-domain.test.mjs

# Build one or all public surfaces
pnpm run build:portfolio
pnpm run build:labs
pnpm run build:timber
pnpm run build

# Build and assemble the unified static release (does not deploy)
pnpm run build:site

# Package already built outputs only
pnpm run site:assemble

# Regenerate the manual separately; requires python-docx and ReportLab
# Inspect the resulting PDF/DOCX before publishing
python scripts/build_complete_manual.py

# Preview outputs
pnpm run preview
pnpm run preview:labs
pnpm run preview:timber
```

Appendix C project control source index

- docs/project-control/STATUS.md - current checkpoint and known risks.
- docs/project-control/DECISIONS.md - locked scope, deployment, privacy, and modeling decisions.
- docs/project-control/EXECUTION_PLAN.md - batch strategy and acceptance sequence.
- docs/project-control/ACCEPTANCE_CRITERIA.md - per-lab, flagship, batch, and final gates.
- docs/project-control/project-manifest.json - canonical 30-lab and 2-flagship inventory.
- docs/project-control/TIMBER_BILLING_SPEC.md - locked measurement and billing rules.
- docs/project-control/PRINTING_ERP_DEMO_SPEC.md - demo/live runtime and acceptance boundary.
- docs/project-control/batches/ - implementation and verification evidence by batch.
- docs/project-control/labs/ - detailed purpose, logic, safety, sources, tests, and limitations for Labs 01-30.
- docs/project-control/redesign-guide-coverage.md - historical August 31 coverage audit and detailed chapter/acceptance template; its pending-content labels predate the expanded manual.

Appendix D known open work at this snapshot

- 1 Finish and verify the portfolio-only UI and motion pass. Do not change Timber, ERP, or Applied Labs applications.
- 2 Verify the locally corrected flagship screenshot and links when the portfolio is separately released; do not redesign the linked applications.
- 3 Detailed Markdown chapters for both flagships and all 30 labs are present. Complete final screenshot and rendered-artifact acceptance before calling the downloadable guide current.
- 4 Reconcile missing later batch evidence using fresh test/build/browser/mobile/console results; do not fill gaps with assumed passes.
- 5 Local manual PDF/DOCX were regenerated on September 4. Complete DOCX visual QA when LibreOffice is available, and verify the current guide PDF in any separately authorized published release.
- 6 Run cross-surface regression, accessibility, route/link, artifact, and console checks, then record the accepted deployment revision/version and workflows.
- 7 ERP's historical dependency findings belong to future separately authorized maintenance, not this portfolio-only task.
- 8 Confirm public repository/resource links and generated resume/guide routes during final release QA. This factual documentation update itself does not change or publish app links/artifacts.

Completing an item means attaching evidence-test result, build result, deployed version, browser check, or updated source-not simply changing its status label.